

VOLUME 05 / RUST BACKEND MASTERY

Rustバックエンド — async・並行処理・ジョブ・運用

Future の仕組みから、タスク・channel・timeout・再試行・ログまで、たくさんの仕事を同時に安全にさばくサーバーの土台を身に付ける

第5巻: Rustバックエンド — async・並行処理・ジョブ・運用

この巻で身に付けること

Web アプリのサーバーには、たくさんの人から同時にリクエストが届きます。ノートアプリ RustNote でも、Aさんがノートを保存している間に、Bさんは一覧を開き、Cさんはログインしようとしています。

1つのリクエストの処理時間のほとんどは、実は**待ち時間**です。DB の返事を待つ、外の API の返事を待つ、ネットワークの向こうからデータが届くのを待つ。待っている間、CPU はほとんど何もしていません。その待ち時間に**ほかの人の仕事を進める**のが、この巻で学ぶ **async**（エイシンク：非同期処理）です。

ただし、たくさんの仕事を同時に進めると、1つずつ順番に進めるときには起きなかった事故が起きます。

- 1件ずつ順番に処理していて、10人が同時に来ると、10人目は**10倍待たされる**
- パスワードのハッシュのような重い計算をしている間、**ほかの全員の応答が止まる**
- 外の API が返事をしないまま、**いつまでも待ち続ける**
- 仕事が一気に押し寄せて、受け付けた仕事でメモリがあふれ、**サーバーごと落ちる**
- 鍵（Mutex）を持ったまま待ってしまい、**全員が鍵を待ったまま動かなくなる**
- サーバーを止めた瞬間に、**保存の途中だった仕事が切れる**
- エラーが起きても、どのリクエストで何が起きたのかを**ログからたどれない**

この巻のテーマは、こうした事故を、**Future**（フューチャー）**・タスク**・**channel**（チャネル）・**Semaphore**（セマフォ）・**timeout**（タイムアウト）・**spawn_blocking**（スポーン・ブロッキング）・**再試行**・**tracing**（トレーシング）という仕組みで防ぐことです。Rust では、多くの間違いを**コンパイルの時点で**止めてくれます。この巻では、そのエラーメッセージを読み解く力も身に付けます。

到達目標

この巻を終えると、次のことができるようになります。

- `async fn` が返す **Future** の正体（「まだ始まっていない仕事の札」で、`poll` されて少しずつ進む状態機械）を説明できる
- `tokio::join!` と `tokio::try_join!` で、複数の待ち時間を重ねて、全体を速くできる
- Tokio の**ランタイム**の役割を説明し、`tokio::spawn` でタスクを作って結果を受け取れる。タスクに `'static` と `Send` が要る理由が分かる
- `JoinSet` と `select!` で、子のタスクをまとめて待ち、最初に終わったものを選べる。同時に動かす数に上限を付けられる

- channel でタスクの間に値を渡し、容量の決まった channel で**多すぎる仕事に「待った」をかけられる**（backpressure：バックプレッシャー）
- `Mutex`・`RwLock`・`Semaphore` で共有するデータと同時に入れる数を守り、「鍵を持ったまま `.await` しない」を守れる
- `timeout` で待つ時間に上限を付け、Ctrl+C を受けたら処理中の仕事を片付けてから止まれる（graceful shutdown：グレースフル・シャットダウン）
- 重い計算を `spawn_blocking` に逃がし、ほかのリクエストを止めない
- HTTP クライアントで外の API を呼び、一時的な失敗だけを間隔をあけて再試行できる。同じ操作を2回送っても困らない形（冪等性：べきとうせい）を説明できる
- tracing で、リクエストごとの情報（何の処理か・誰の処理か・何ミリ秒かかったか）を付けたログを出せる
- 画面の外で動く仕事（バックグラウンドジョブ）を、サーバーが落ちても続けられる形で設計できる

学習地図

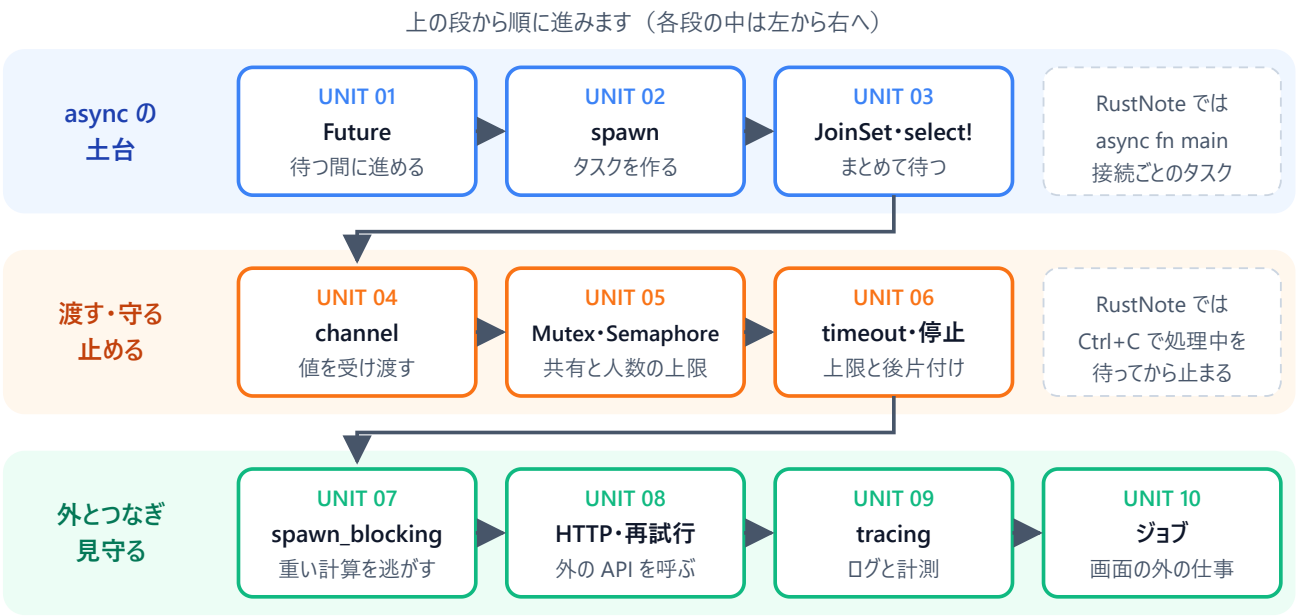


図5-1: 第5巻の学習地図。async の土台から始めて、タスクの間の受け渡し・守り方・止め方、外とのやり取りと見守り方へ進む

単元	テーマ	ひとことで言うと
UNIT 01	async・Future・await の正体	「待つ間に、ほかの仕事を進める」仕組みを、中からのぞく
UNIT 02	Tokio のランタイム・spawn・JoinHandle	仕事（タスク）を作り、何人もの作業員で分け合う
UNIT 03	JoinSet・select!・同時に動かす数の管理（準備中）	子のタスクをまとめて待ち、最初に終わったものを選ぶ
UNIT 04	channel・backpressure・所有権の移動（準備中）	タスクの間で値を受け渡し、多すぎる仕事に「待った」をかける
UNIT 05	Mutex・RwLock・Semaphore と .await（準備中）	共有するデータと、同時に入れる数を守る
UNIT 06	timeout・キャンセル・graceful shutdown（準備中）	待つ時間に上限を付け、止めるときは仕事を片付けてから止める
UNIT 07	spawn_blocking と重い計算（準備中）	パスワードのハッシュのような重い仕事を、別の場所に逃がす
UNIT 08	HTTP クライアント・再試行・冪等性（準備中）	外の API を呼び、失敗したら賢くやり直す
UNIT 09	tracing・構造化ログ・メトリクス（準備中）	何が起きたかを、あとからたどれるように記録する
UNIT 10	バックグラウンドジョブと運用（準備中）	画面の外で動く仕事を、落ちても続けられる形にする
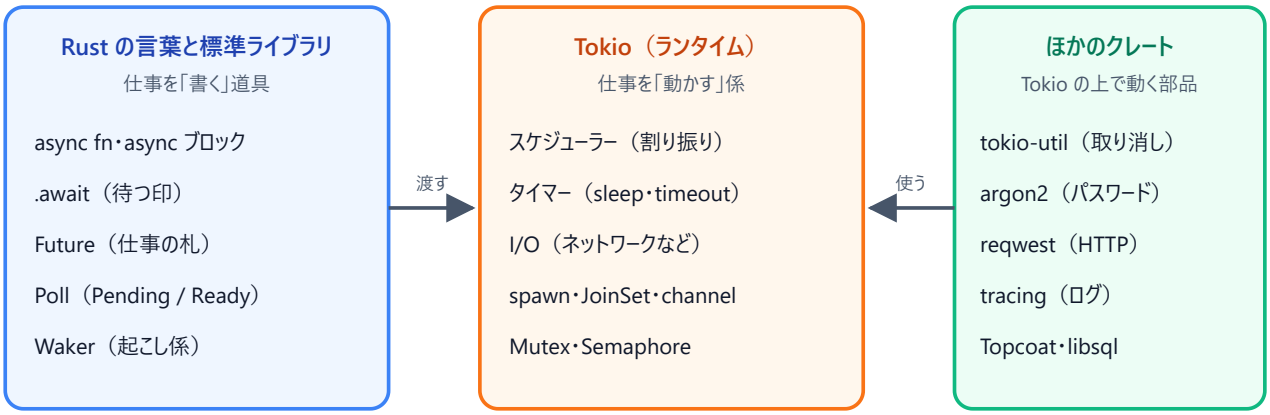
> **お知らせ:** この版には UNIT 01・02 を収録しています。UNIT 03〜10 は準備中で、次の更新から順に追加します。本文の中の「UNIT 03 で学びます」のような予告は、これから追加する単元を指しています。

この巻に出てくる名前の整理

この巻には、Rust の言葉そのものと、クレート（部品）の名前が混ざって出てきます。最初に、どれが何なのかを分けておきましょう。

- `async` / `.await` / `Future`: Rust の言葉と標準ライブラリにある仕組みです。「待ちながら進む仕事」を**書く**ための道具で、どの Rust のプログラムでも使えます。
- **Tokio**（トーキョー）: その仕事を**実際に動かす**ためのクレートです。Rust の標準ライブラリには「Future を動かす係」が入っていないので、Tokio のような**ランタイム**を足して使います。RustNote が使う Topcoat も、第7巻の libsql も、Tokio の上で動きます。

- **そのほかのクレート**: Tokio の上で動く、目的別の部品です。この巻では、取り消しの合図に tokio-util、パスワードに argon2、HTTP クライアントに reqwest（リクエスト）、ログに tracing を使います。



async fn が作った Future を Tokio が動かし、ほかのクレートも Tokio の力を借りて動く

図5-2: この巻に出てくる名前の関係。仕事を「書く」のは Rust の言葉、「動かす」のは Tokio、目的別の部品は Tokio の上で動く

> **ヒント:** 第7巻（Turso・libSQL）では、`#[tokio::main]` を「`main` の中で `.await` を使えるようにするおまじない」、`.await` を「返事が来るまで待つ印」として使いました。その「おまじない」の中身を、UNIT 01 と UNIT 02 で開けて見ていきます。第7巻をまだ読んでいなくても、この巻は読み進められます。

前提知識（先に読んでおく巻）

- **第1巻（環境構築）**: Rust・VS Code・rust-analyzer がインストール済みで、`cargo new` と `cargo run` ができること。
- **第2巻（Rust 基礎）**: `struct` / `enum` / `match` / `Option` / `Result` / `?` / `Vec` / `String` / クロージャの基本。
- **第3巻（所有権・借用）**: 所有権の移動（move）と借用。`'static`（第3巻 UNIT 06）。そして特に、**第3巻 UNIT 10 の** `Arc`・`Mutex`・`Send`・`Sync` は、この巻で何度も使います。スレッドを作る `std::thread::spawn` の話を覚えていると、UNIT 02 の `tokio::spawn` がすっと入ります。
- **第4巻（設計）**: trait と generics、エラー型（thiserror と anyhow）、`Cargo.toml` の版の書き方（第4巻 UNIT 07）。

> **注意:** この巻では、スレッド・`Arc`・`Mutex` を「第3巻で学んだもの」として使います。`Send` と `Sync` の意味があいまいなときは、先に第3巻 UNIT 10 の「図で理解する」と「よくあるエラー」だけでも読み返しておくと、UNIT 02 のエラーメッセージがずっと読みやすくなります。

この巻の進め方

各単元では、次の順番を1サイクルとして進めてください。async は、頭の中で考えるだけでは、どの順番で何が起きるのかが想像しにくい分野です。**表示の順番を予想してから動かす**のが、いちばんの近道です。

1. **予想する:** 実行する前に、どの行がどの順番で表示されるか、全体で何秒かかるかを予想します。
2. **そのまま動かす:** 教材のコードをそのまま実行し、結果が教材と同じか比べます。
3. **1か所だけ変える:** 待ち時間や `.await` の位置を1つだけ変えて、表示の順番と時間がどう変わるかを観察します。「よくあるエラー」の節のコードも実際にビルドして、エラーメッセージを自分の目で読みます。
4. **整える:** 単元の終わりには、次の3つのコマンドを実行する習慣をつけます。

```
# コードの見た目を Rust の標準の形に整える
cargo fmt
# よくある書き間違い・もったいない書き方を指摘してもらう
cargo clippy
# テストを実行する
cargo test
```

> **ヒント:** この巻の実行結果には、かかった時間（ `0.5 秒` など）や、スレッドの番号（ `ThreadId(13)` など）が出てきます。時間はパソコンの速さやその時の混み具合で少し変わり、スレッドの番号は実行するたびに変わります。表示の順番が教材と入れ替わることもあります。どういうときに入れ替わるのかは、UNIT 02 で説明します。

使うツールと版

この巻のコードは、すべて次の環境で実際にコンパイル・実行し、実行結果とエラーメッセージは実際の出力をそのまま載せています（パソコンのユーザー名などを含む部分だけ `...` に置き換えています）。

項目	版・内容
OS	Windows 11（ARM64）。コマンドは PowerShell を主に、WSL / macOS / Linux は補足で示します
Rust	**1.98 以上**（本書シリーズの基準）。動作確認は rustc 1.99.0 / cargo 1.99.0（`aarch64-pc-windows-msvc`）
edition	2024（`cargo new` の既定値）
tokio	1.53.3（`full` の機能つき。この巻の全単元で使う）
tokio-util	0.7.20（UNIT 06。取り消しの合図と、タスクの見張り）
argon2	0.6.0（UNIT 07。パスワードのハッシュ）
reqwest	0.13.5（UNIT 08。HTTP クライアント。ARM64 版の Windows では、ビルドに clang が必要）
serde / serde_json	1.0.229 / 1.0.154（UNIT 08・09。JSON との変換）
tracing / tracing-subscriber	0.1.44 / 0.3.23（UNIT 09。ログ）

UNIT 01 と UNIT 02 で使うクレートは tokio だけです。ほかのクレートは、使う単元で `cargo add` します。

練習用プロジェクトを用意する

この巻では、練習用のプロジェクト `async_lab` を1つ作り、小さなプログラムを `src/bin/` に1つずつ足していきます。

```
cargo new async_lab
cd async_lab
mkdir src/bin
cargo add tokio@=1.53.3 --features full
```

`cargo new` と `cargo add` の実際の出力は次のとおりです。

```
    Creating binary (application) `async_lab` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-
lang.org/cargo/reference/manifest.html
    Updating crates.io index
      Adding tokio =1.53.3 to dependencies
             Features:
             + bytes
             + fs
             + full
             + io-std
             + io-util
             + libc
             + macros
             + net
             + parking_lot
             + process
             + rt
             + rt-multi-thread
             + signal
             + signal-hook-registry
             + socket2
             + sync
             + time
             + tokio-macros
             - io-uring
             - mio
             - schedule-latency
             - taskdump
             - test-util
             - tracing
             - windows-sys
    Updating crates.io index
     Locking 24 packages to highest Rust 1.99.0 compatible versions
      Adding tokio v1.53.3 (available: v1.54.0)
```

- `tokio@=1.53.3` の `=` は「ちょうど 1.53.3」という意味です（第4巻 UNIT 07）。最後の行の `available: v1.54.0` は「もっと新しい 1.54.0 も出ています」というお知らせです。`=` を付けずに `tokio@1.53.3` と書くと、Cargo は 1.x の中でいちばん新しい版を選ぶので、2026年10月22日の時点では 1.54.0 が入ります。この巻の出力と同じ結果になるように、版を固定しておきます。
- `--features full` は、Tokio の機能を**全部** ON にする指定です。`+` が付いた機能が ON になりました。`time`（待つ・時間の上限）、`sync`（channel や Mutex）、`macros`（`#[tokio::main]`）、`rt-multi-thread`（何本ものスレッドで動かす）、`signal`（Ctrl+C を受け取る）など、この巻で使う機能が全部入っています。
- 本番のアプリでは、使う機能だけを選ぶのがふつうです。RustNote の `Cargo.toml` は `features = ["rt-multi-thread", "macros"]` の2つだけです。機能を絞ると、ビルドが速くなり、使わない部品が入りません。

最後に、`Cargo.toml` の `[package]` の中、`edition = "2024"` の次の行に `rust-version = "1.98"` を手で1行足します。できあがった `Cargo.toml` は次のとおりです。

```
[package]
name = "async_lab"
version = "0.1.0"
edition = "2024"
rust-version = "1.98"

[dependencies]
tokio = { version = "=1.53.3", features = ["full"] }
```

一度 `cargo build` しておきましょう。最初のビルドでは Tokio とその部品をコンパイルするので、少し時間がかかります（この PC では約12秒でした）。

```
   Compiling tokio v1.53.3
   Compiling async_lab v0.1.0 (...\async_lab)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 11.89s
```

src/bin/ に置いたファイルは、それぞれが1つのプログラム

`src/bin/` フォルダに置いた `.rs` ファイルは、1つずつが別のプログラムになります。たとえば `src/bin/u01_lazy.rs` を作ったら、次のように名前（拡張子なし）を指定して実行します。

```
cargo run -q --bin u01_lazy
```

- `--bin u01_lazy` は「`src/bin/u01_lazy.rs` のプログラムを実行して」という指定です。
- `-q`（quiet：静かに）を付けると、`Compiling ...` や `Running ...` のような Cargo の表示が消えて、プログラムが表示した内容だけが残ります。この巻の実行結果は、特に断りがなければ `-q` を付けて実行したものです。
- コンパイルエラーの例は、エラーの全文を見るために `-q` を付けずに `cargo build --bin 名前` や `cargo run --bin 名前` で実行しています。

> **ヒント:** VS Code で `src/bin/` のファイルを開くと、`fn main` の上に「▶ Run」の文字が出ます。これをクリックしても同じように実行できます（rust-analyzer の機能です）。

UNIT 01: async・Future・await の正体

この単元のゴール

- `async fn` を呼んだだけでは中身が動かず、**Future**（フューチャー）という「仕事の札」が返ってくること、`.await` して初めて動き出すことを、実行結果で説明できる
- `.await` を順番に書いたときと、`tokio::join!` で同時に進めたときの時間の違いを、図で説明できる
- `tokio::try_join!` で、どれか1つが失敗したら、残りを待たずにすぐ失敗を返す処理を書ける
- Future が **poll**（ポール）されて、`Pending`（まだです）と `Ready`（できました）を返しながら少しずつ進む仕組みを、小さな実行係を自分で動かして確かめられる
- `async` や `.await` の付け忘れで出るエラーと警告を読んで、直せる

たとえ話でつかむ

ラーメン屋の店員にたとえます。麺をゆでるには3分かかります。

- **順番に待つ店員**は、麺を鍋に入れたら、ゆで上がるまでの3分間、鍋の前にじっと立っています。その間に来たお客さんは、注文すらできずに待たされます。
- **待ち時間をうまく使う店員**は、麺を鍋に入れたらタイマーを3分にセットして、次のお客さんの注文を取りに行きます。タイマーが鳴ったら、鍋に戻って麺を上げます。

店員は1人のままなのに、後の店員のほうが、ずっとたくさんのお客さんをさばけます。待ち時間が重なっているからです。これが **async**（非同期処理）の考え方です。この店のしくみを、Rust の言葉に置き換えると次のようになります。

ラーメン屋	Rust の async
注文の伝票	**Future**。書いた（作った）だけでは、料理は始まらない
店員	**ランタイム**（実行係）。伝票を手に取って、進められるところまで進める
伝票を手に取って進めること	**poll**（ポール）
「まだゆでている」	`Poll::Pending`。店員は伝票を置いて、ほかの伝票を進める
「できあがり」	`Poll::Ready(値)`
キッチンタイマー	**Waker**（ウェイカー：起こし係）。鳴ったら「あの伝票をもう一度見て」と店員に知らせる
伝票に書かれた「ここで待つ」の印	`.await`。待つ間、店員はほかの伝票を進めてよい

> **用語:** 非同期処理（async：エイシンク。待ち時間の間に、ほかの仕事を進められるように書いた処理。Rust では `async fn` や `async { }` で書く）

> **用語:** Future（フューチャー：「まだ終わっていない仕事」を表す値。`async fn` を呼ぶと返ってくる。`.await` されるまで、中身は動かない）

> **用語:** poll（ポール：Future に「進められるところまで進めて」と頼むこと。答えは `Poll::Pending`（まだです）か `Poll::Ready(値)`（できました）のどちらか）

> **用語:** ランタイム（runtime：Future を poll して動かす係。executor（エグゼキューター：実行係）とも呼ぶ。Rust の標準ライブラリには入っていないので、この巻では Tokio を使う）

図で理解する

まず、「順番に待つ」と「待ち時間を重ねる」で、全体の時間がどう変わるかを見ます。0.5 秒かかる DB への問い合わせを2つ行う場合です。この後の例1-2で、実際に測ります。

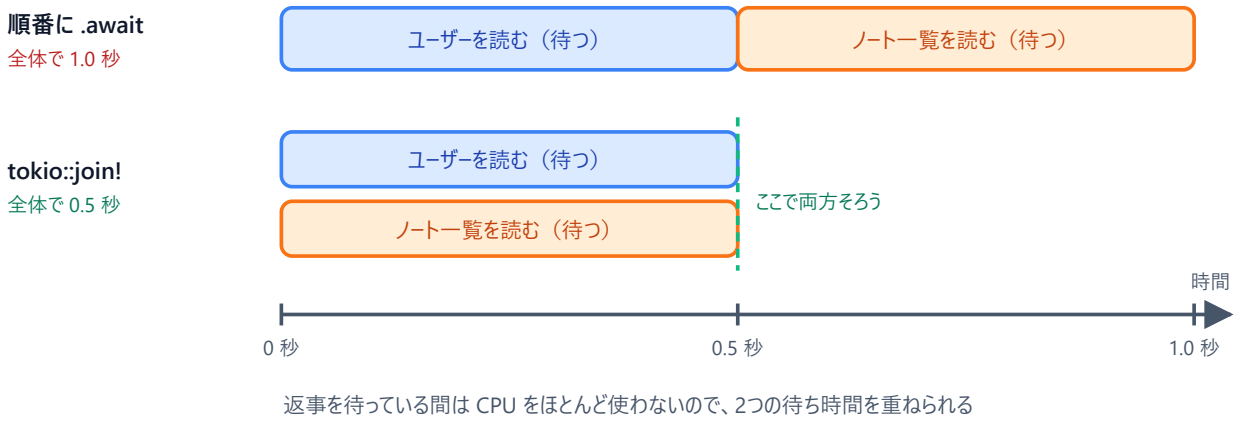


図5-3: 0.5 秒の問い合わせを2つ行うときの時間。順番に .await すると足し算で 1.0 秒、tokio::join! で同時に進めると、長いほうの 0.5 秒で終わる

- 順番に `.await` すると、1つ目の返事が来るまで、2つ目の問い合わせは**始まりもしません**。全体の時間は足し算になります。
- `tokio::join!` を使うと、2つの問い合わせを両方とも始めてから、両方の返事を待ちます。全体の時間は、**いちばん長いもの**で決まります。

> **用語:** 並行と並列（並行 concurrency：コンカレンシー は、複数の仕事を「切り替えながら」少しずつ進めること。並列 parallelism：パラレリズム は、複数の CPU で「本当に同時に」動かすこと。`tokio::join!` は並行で、CPU は1つしか使わない）

次に、Future が中でどう動いているかです。ランタイムは Future を poll し、Future は「まだです」か「できました」を答えます。

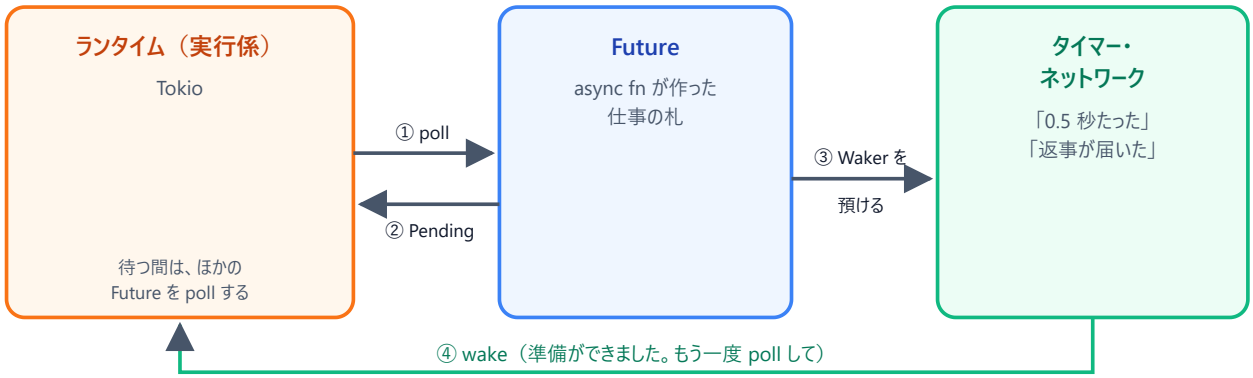


図5-4: Future が進むしくみ。poll されて「まだ」なら Waker（起こし係）を預けて Pending を返し、準備ができたら起こしてもらって、もう一度 poll される

- ① ランタイムが Future を poll します。② 返事がまだなら、Future は `Pending` を返します。③ そのとき、「準備ができたら起こしてください」と **Waker** をタイマーやネットワークの係に預けておきます。
- ランタイムは、その Future のことはいったん忘れて、ほかの Future を poll します。待っている Future のために、CPU を使い続けることはありません。
- ④ 時間がたったり返事が届いたりすると、Waker が「もう一度 poll して」とランタイムに知らせます。⑤ もう一度 poll すると、今度は `Ready(値)` が返ります。

> **ヒント:** ふだんのプログラムでは、自分で poll を呼ぶことはありません。`.await` と書けば、ここまでの流れを Rust とランタイムが全部やってくれます。例1-4で、この流れを一度だけ自分の目で確かめます。

まず動かす最小コード

例1-1: async fn を呼んだだけでは、中身は動かない

> **▼ このコードがやること（先に日本語で）:** 「お湯を沸かす」非同期関数 `boil_water` を作ります。`main` から呼んだ直後と、`.await` した後に、それぞれ表示を出して、関数の中身が**いつ**動いたのかを確かめます。

```rust
use std::time::Duration;

// 「お湯を沸かす」非同期関数。呼ぶと Future（まだ始まっていない仕事の札）が返る
async fn boil_water() -> u32 {
    println!("  [boil_water] 火をつけました");
    // 0.3 秒待つ（この間、ほかの仕事に席を譲る）
    tokio::time::sleep(Duration::from_millis(300)).await;
    println!("  [boil_water] 沸きました");
    100
}

#[tokio::main]
async fn main() {
    println!("1. boil_water() を呼びます");
    let future = boil_water();
    println!("2. 呼び終わりました（まだ火はついていません）");
    let temperature = future.await;
    println!("3. .await が終わりました: {temperature} 度");
}
```

`src/bin/u01_lazy.rs` として保存して実行します。

```
cargo run -q --bin u01_lazy
```

▼ 実行結果

```
1. boil_water() を呼びます
2. 呼び終わりました（まだ火はついていません）
  [boil_water] 火をつけました
  [boil_water] 沸きました
3. .await が終わりました: 100 度
```

`boil_water()` を呼んだのは「1.」と「2.」の間なのに、「火をつけました」は「2.」の**後**に表示されています。

▼ コードを1つずつ分解して解説

解説1: async fn を呼ぶと、戻り値の代わりに Future が返る

- `async fn boil_water() -> u32` は「`u32` を返す関数」ではなく、「**いつか `u32` ができあがる Future を返す関数**」です。
- `let future = boil_water();` の時点では、関数の中身は1行も動いていません。手元に「お湯を沸かす」と書かれた伝票ができただけです。
- だから「2.」の表示が先に出ます。これを、Future は**怠け者**（lazy：レイジー）だ、と言います。

解説2: .await して初めて、中身が動き出す

- `future.await` と書いたところで、ランタイムが伝票を手に取り、中身を進め始めます。「火をつけました」はここで表示されます。
- 中身の `sleep(...).await` で 0.3 秒待つ間、`main` も一緒に止まって待ちます。中身が最後まで進んで `100` を返すと、`future.await` 全体が `100` という値になり、`temperature` に入ります。

> **ヒント:** JavaScript の `async` 関数は、呼んだ瞬間に中身が動き始めます（Promise）。Rust の Future は `.await` するまで動きません。JavaScript を知っている人ほど、ここでつまずきやすいので注意しましょう。

解説3: tokio::time::sleep は「席を譲って」待つ

- `tokio::time::sleep(時間)` は、指定した時間だけ待つ Future を作ります。`.await` すると、待っている間、**ほかの Future に席を譲ります。**
- 第3巻で使った `std::thread::sleep` は、スレッドそのものを眠らせます。async の中でこれを使うと、そのスレッドで動いているほかの Future も全部止まってしまいます。

> **注意:** `async fn` の中では、`std::thread::sleep` を使わず、`tokio::time::sleep(...).await` を使います。同じように、時間のかかる計算やファイルの読み書きも、そのまま書くとほかの仕事を止めてしまいます。その避け方は UNIT 07 で学びます。

解説4: #[tokio::main] は、main の外側にランタイムを用意する

- Rust の `main` 関数は、ふつうは `async` にできません（「よくあるエラー1」で確かめます）。`main` を呼ぶのは OS で、OS は Future の poll のしかたを知らないからです。
- `#[tokio::main]` を付けると、「Tokio のランタイムを作って、その上で `async fn main` の Future を最後まで動かす」ふつうの `main` に書き換えてくれます。中で何をしているかは、UNIT 02 で自分の手で書いて確かめます。

例1-2: 順番に待つか、同時に待つか

> **▼ このコードがやること（先に日本語で）:** 0.5 秒かかる「DB への問い合わせ」の代わりの関数 `query` を2回使います。①では1つずつ順番に `.await` し、②では `tokio::join!` で2つを同時に進めて、それぞれ全体で何秒かかったかを表示します。

```rust
use std::time::{Duration, Instant};

// DB への問い合わせの代わり: 指定した時間だけ待ってから結果を返す
async fn query(name: &str, millis: u64) -> String {
    tokio::time::sleep(Duration::from_millis(millis)).await;
    format!("{name} の結果")
}

#[tokio::main]
async fn main() {
    // ① 順番に .await する（1つ終わってから次を始める）
    let start = Instant::now();
    let user = query("ユーザー", 500).await;
    let notes = query("ノート一覧", 500).await;
    println!(
        "順番に:   {user} / {notes} / {:.1} 秒",
        start.elapsed().as_secs_f64()
    );

    // ② tokio::join! で同時に進める（両方の待ち時間が重なる）
    let start = Instant::now();
    let (user, notes) = tokio::join!(query("ユーザー", 500), query("ノート一覧", 500));
    println!(
        "同時に:   {user} / {notes} / {:.1} 秒",
        start.elapsed().as_secs_f64()
    );
}
```

`src/bin/u01_join.rs` として保存して実行します。

```
cargo run -q --bin u01_join
```

▼ 実行結果

```
順番に:   ユーザー の結果 / ノート一覧 の結果 / 1.0 秒
同時に:   ユーザー の結果 / ノート一覧 の結果 / 0.5 秒
```

▼ コードを1つずつ分解して解説

解説1: 順番に .await すると、時間は足し算になる

- `query("ユーザー", 500).await` は、0.5 秒待って結果を受け取るまで、次の行へ進みません。2つ目の `query` は、1つ目が終わってから**作られ**、そこから 0.5 秒待ちます。
- だから全体で 0.5 + 0.5 = 1.0 秒です。

- `Instant::now()` は「今の時刻」、`start.elapsed()` は「`start` からたった時間」です。`as_secs_f64()` で秒の小数にし、`{:.1}` で小数第1位まで表示しています。

解説2: tokio::join! は、全部の Future を同時に進めて、全部を待つ

- `tokio::join!(A, B)` は、A と B の両方を進め、**両方が終わったら**、結果を `(Aの結果, Bの結果)` のタプルで返します。
- 中では、A を poll して `Pending` なら B を poll し、どちらかが起こされたらまた poll し…を繰り返しています。2つの 0.5 秒の待ちが重なるので、全体で 0.5 秒です。
- 3つ以上でも `tokio::join!(A, B, C)` のように並べられます（練習問題2）。

解説3: join! は「並行」。CPU を2つ使うわけではない

- `tokio::join!` は、1つの流れ（1つのタスク）の中で、A と B を**切り替えながら**進めます。CPU は1つしか使っていません。
- 速くなったのは、2つとも「待っているだけ」だったからです。もし A と B が、待ち時間のない重い計算だったら、`join!` にしても速くなりません。計算を本当に同時に（並列に）動かす方法は、UNIT 02 と UNIT 07 で学びます。

解説4: 前の結果を使うときは、順番に待つしかない

- 「ユーザーを読んでから、**そのユーザーの id で**ノートを読む」のように、2つ目が1つ目の結果を使うなら、`join!` にはできません。1つ目が終わらないと、2つ目を始められないからです。
- `join!` が使えるのは、**お互いに関係のない待ち**だけです。コードを読むときは「この2つは、どちらかの結果を使っているか？」を確かめましょう。

もう一歩: 実務に近い形

例1-3: 1つのリクエストで、2つの問い合わせを同時に進める（try_join!）

> **▼ このコードがやること（先に日本語で）**：画面の上に「hanako さんのノートは 3 件」と出すダッシュボードを考えます。ユーザーを読む問い合わせ（0.3 秒）と、ノートを数える問い合わせ（0.5 秒）を同時に進めます。どちらかが失敗したら、もう片方を待たずに、すぐ失敗を返します。いるユーザー（1）と、いないユーザー（2）で試します。

```rust
use std::time::{Duration, Instant};

#[derive(Debug)]
struct User {
    name: String,
}

// ユーザーを読む（DB の代わり）。0.3 秒かかる
async fn load_user(user_id: i64) -> Result<User, String> {
    tokio::time::sleep(Duration::from_millis(300)).await;
    if user_id == 1 {
        Ok(User {
            name: String::from("hanako"),
        })
    } else {
        Err(format!("ユーザー {user_id} はいません"))
    }
}

// ノートの件数を数える（DB の代わり）。0.5 秒かかる
async fn count_notes(user_id: i64) -> Result<u32, String> {
    tokio::time::sleep(Duration::from_millis(500)).await;
    Ok(user_id as u32 * 3)
}

// 1つのリクエストの処理: 2つの問い合わせを同時に進め、どちらかが失敗したらすぐ失敗を返す
async fn dashboard(user_id: i64) -> Result<String, String> {
    let (user, count) = tokio::try_join!(load_user(user_id), count_notes(user_id))?;
    Ok(format!("{} さんのノートは {count} 件", user.name))
}

#[tokio::main]
async fn main() {
    for user_id in [1, 2] {
        let start = Instant::now();
        let result = dashboard(user_id).await;
        println!("{result:?} / {:.1} 秒", start.elapsed().as_secs_f64());
    }
}
```

`src/bin/u01_try_join.rs` として保存して実行します。

```
cargo run -q --bin u01_try_join
```

▼ 実行結果

```
Ok("hanako さんのノートは 3 件") / 0.5 秒
Err("ユーザー 2 はいません") / 0.3 秒
```

ユーザー 2 の失敗は 0.3 秒で返ってきました。ノートを数える 0.5 秒を待っていません。

▼ コードを1つずつ分解して解説

解説1: try_join! は、Result を返す Future のための join!

- `tokio::try_join!(A, B)` の A と B は、どちらも `Result` を返す Future です。
- 両方が `Ok` なら、`Ok((Aの中身, Bの中身))` を返します。どれか1つでも `Err` になったら、**その時点で**その `Err` を返します。
- 後ろの `?` で、`Err` なら `dashboard` からそのエラーを返し、`Ok` なら中身のタプルを取り出して `(user, count)` に分けています（第2巻 UNIT 09 の `?` と同じです）。

解説2: 失敗したら、残りの Future は途中で捨てられる

- ユーザー 2 では、0.3 秒の時点で `load_user` が `Err` を返しました。`try_join!` はすぐに `Err` を返し、まだ途中だった `count_notes` の Future は **drop**（ドロップ：片付け）されます。
- drop された Future は、二度と poll されません。つまり `count_notes` は、0.3 秒の時点で**途中で止まった**ことになります。
- これは便利な反面、「途中まで書き込んだところで止まる」と困る処理もあります。Future が途中で捨てられることを**キャンセル**と言い、UNIT 06 でくわしく扱います。

解説3: エラーの型は、そろえる必要がある

- `try_join!` に渡す Future は、エラーの型が同じでなければなりません。ここでは、どちらも `Result<_, String>` にそろえています。
- 実際のアプリでは、DB のエラーと検証のエラーのように型が違うことがよくあります。そのときは、第4巻 UNIT 05 で作ったような、アプリ共通のエラー型（RustNote の `AppError` など）に変換してから渡します。

例1-4: 小さな実行係を自分で作って、poll の様子をのぞく

ここは、Future の中身をのぞく発展の内容です。ふだんのアプリで、自分で Future を実装したり poll を呼んだりすることは、ほとんどありません。難しく感じたら、実行結果の流れだけ眺めて、例1-5へ進んでもかまいません。

> **▼ このコードがやること（先に日本語で）**：決まった回数だけ「まだです」と答える Future `Countdown` を自分で作ります。それを2回 `.await` する `make_tea` を、Tokio を使わずに、自分で作った最小の実行係 `block_on` で動かします。poll が何回呼ばれ、そのたびにどこまで進むかを表示します。

```rust
use std::future::Future;
use std::pin::{Pin, pin};
use std::task::{Context, Poll, Waker};

// 決まった回数だけ「まだです（Pending）」と答え、その後「できました（Ready）」と答える Future
struct Countdown {
    remaining: u32,
}

impl Future for Countdown {
    type Output = &'static str;

    fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output> {
        if self.remaining == 0 {
            println!("    Countdown: Ready を返す");
            Poll::Ready("完了")
        } else {
            println!(
                "    Countdown: Pending を返す（あと {} 回）",
                self.remaining
            );
            self.remaining -= 1;
            // 「準備ができたので、もう一度 poll してください」と実行係に知らせる
            cx.waker().wake_by_ref();
            Poll::Pending
        }
    }
}

// async fn の中に .await が2つある。コンパイラはこれを状態機械に変換する
async fn make_tea() -> String {
    println!("  [make_tea] 開始");
    let first = Countdown { remaining: 2 }.await;
    println!("  [make_tea] 1つ目の待ちが終わった: {first}");
    let second = Countdown { remaining: 1 }.await;
    println!("  [make_tea] 2つ目の待ちが終わった: {second}");
    String::from("お茶")
}

// 最小の実行係（executor）: Ready が返るまで poll を呼び続ける
fn block_on<F: Future>(future: F) -> F::Output {
    // poll するには、Future が動かない場所に固定（pin）されている必要がある
    let mut future = pin!(future);
    // 何もしない Waker（起こし係）。この実行係はどうせすぐ次の poll をする
    let mut cx = Context::from_waker(Waker::noop());
    let mut count = 0;
    loop {
        count += 1;
        println!("実行係: {count} 回目の poll");
```

```
        if let Poll::Ready(value) = future.as_mut().poll(&mut cx) {
            return value;
        }
    }
}

fn main() {
    let tea = make_tea();
    println!("make_tea() を呼んだ。まだ何も表示されていないはず");
    let result = block_on(tea);
    println!("結果: {result}");
}
```

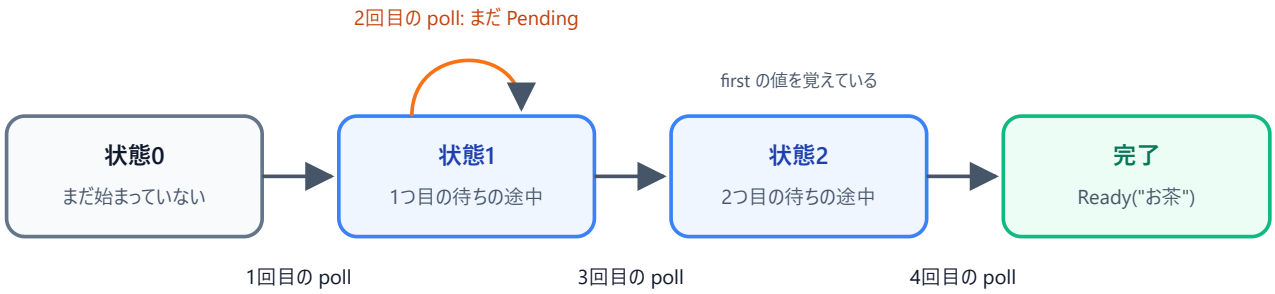
`src/bin/u01_mini_executor.rs` として保存して実行します。このプログラムは Tokio を使っていません（`#[tokio::main]` もありません）。

```
cargo run -q --bin u01_mini_executor
```

▼ 実行結果

```
make_tea() を呼んだ。まだ何も表示されていないはず
実行係: 1 回目の poll
  [make_tea] 開始
    Countdown: Pending を返す（あと 2 回）
実行係: 2 回目の poll
    Countdown: Pending を返す（あと 1 回）
実行係: 3 回目の poll
    Countdown: Ready を返す
  [make_tea] 1つ目の待ちが終わった: 完了
    Countdown: Pending を返す（あと 1 回）
実行係: 4 回目の poll
    Countdown: Ready を返す
  [make_tea] 2つ目の待ちが終わった: 完了
結果: お茶
```

`make_tea` は、コンパイラによって次のような「状態機械」に作り変えられています。poll されるたびに、今の状態から次の `Pending` まで一気に進みます。



3回目の poll は、1つ目の待ちを終えて、2つ目の待ちの Pending まで一気に進んだ

図5-5: make_tea が作り変えられた状態機械。poll されるたびに、今の状態から次の Pending まで進み、.await をまたいで使う変数（first）は状態の中に保存される

▼ コードを1つずつ分解して解説

解説1: Future は、poll という関数を1つ持つ trait

- `Future` は標準ライブラリにある trait（第4巻 UNIT 02）で、`poll` という関数を1つだけ持っています。`type Output` は、できあがったときの値の型です。
- `poll` の答えは `Poll::Pending`（まだです）か `Poll::Ready(値)`（できました）のどちらかです。
- `Countdown` は、`remaining` が 0 になるまで `Pending` を返し、0 になったら `Ready("完了")` を返します。

解説2: Pending を返すときは、必ず「起こす約束」をする

- `cx.waker().wake_by_ref();` は、「もう一度 poll してください」とランタイムに知らせる呼び出しです。
- `Countdown` は「すぐに準備ができる」ので、その場で起こしています。本物のタイマーなら、「時間になったら起こす」ように Waker をタイマーに預けます（図の③）。
- `Pending` を返すのに、誰も Waker で起こさないと、その Future は**二度と poll されず、止まったまま**になります。自分で Future を実装するときに、いちばん気をつける約束です。

解説3: block_on は「Ready が返るまで poll し続ける」だけの実行係

- `pin!(future)` は、Future をメモリの中で**動かないように留めて**います（pin：ピン留め）。async fn が作る状態機械は、自分の中の変数を指す参照を持つことがあるので、poll の途中で場所が変わると困るからです。`poll` の引数が `Pin<&mut Self>` になっているのは、この約束のためです。
- `Waker::noop()` は、何もしない Waker です。この実行係は、`Pending` が返っても**すぐに次の poll をする**ので、起こしてもらう必要がありません。
- 本物のランタイム（Tokio）は、こんなふうにぐるぐる poll し続けたりはしません。`Pending` を返した Future は脇に置いて、ほかの Future を進め、**Waker で起こされた Future だけ**をもう一度 poll します。だから、待っている Future がいくつあっても CPU を無駄に使いません。

解説4: async fn は、.await のところで区切られた状態機械になる

- 実行結果の「3 回目の poll」を見てください。1つ目の `Countdown` が `Ready` を返すと、`make_tea` は止まらずにそのまま次の行へ進み、2つ目の `Countdown` が `Pending` を返したところで、ようやく poll から戻っています。

- つまり、1回の poll は「次の `Pending` まで」進みます。`.await` は、そこで止まるかもしれない**区切り**です。
- `.await` の後でも使う変数（ここでは `first`）は、止まっている間も覚えておかなければならないので、状態機械の中に保存されます。次の例1-5で、それを Future の大きさで確かめます。

例1-5: .await をまたぐ変数は、Future の中にしまわれる

> **▼ このコードがやること（先に日本語で）**：1KB（1024 バイト）の配列を、`.await` **の後**でも使う関数と、`.await` **の前**に使い終わる関数を作ります。それぞれが返す Future の大きさを `size_of_val` で測って比べます。

```rust
use std::mem::size_of_val;
use std::time::Duration;

async fn wait() {
    tokio::time::sleep(Duration::from_millis(10)).await;
}

// 1KB の配列を作り、.await の「後」でも使う
async fn keeps_buffer() -> usize {
    let buffer = [1u8; 1024];
    wait().await;
    buffer.len()
}

// 1KB の配列は .await の「前」に使い終わる
async fn uses_buffer_before_await() -> usize {
    let len = {
        let buffer = [1u8; 1024];
        buffer.len()
    };
    wait().await;
    len
}

#[tokio::main]
async fn main() {
    let a = wait();
    let b = keeps_buffer();
    let c = uses_buffer_before_await();
    println!(
        "wait() の Future:                    {} バイト",
        size_of_val(&a)
    );
    println!(
        "keeps_buffer() の Future:            {} バイト",
        size_of_val(&b)
    );
    println!(
        "uses_buffer_before_await() の Future: {} バイト",
        size_of_val(&c)
    );
    println!("結果: {} {} {:?}", b.await, c.await, a.await);
}
```

`src/bin/u01_future_size.rs` として保存して実行します。

```
cargo run -q --bin u01_future_size
```

▼ 実行結果

```
wait() の Future:                    120 バイト
keeps_buffer() の Future:            1152 バイト
uses_buffer_before_await() の Future: 136 バイト
結果: 1024 1024 ()
```

▼ コードを1つずつ分解して解説

解説1: .await の後で使う配列は、Future の中に入る

- `keeps_buffer` の Future は 1152 バイトで、`wait()` の Future（120 バイト）より 1KB ほど大きくなっています。`.await` で止まっている間も、後で `buffer.len()` を使うために、配列を Future の中に取っておく必要があるからです。
- `uses_buffer_before_await` では、配列は `{ }` の中で使い終わって消えます。`.await` のときには配列はもう無いので、Future の中には `len`（数字1つ）しか入らず、136 バイトで済んでいます。
- 数字の細かい内訳（何バイトが何に使われるか）は、Rust の版によって変わることがあります。覚えるのは「**`.await` をまたいで使う変数は、Future の中にしまわれる**」という一点です。

解説2: この性質が、UNIT 02 のエラーの正体になる

- `.await` をまたいで持っている値は、Future の一部です。UNIT 02 では、Future を別のスレッドに渡して動かします。そのとき、Future の中に「スレッドをまたげない値」（第3巻 UNIT 10 の `Rc` など）がしまわれていると、コンパイルエラーになります。
- 大きな配列を持ったまま `.await` すると、Future も大きくなります。大きなデータは `Vec` や `Box` にすると、本体はヒープに置かれ、Future の中には小さな「持ち手」（`Vec` なら 24 バイト）だけが入ります。

> **ヒント:** `size_of_val` は、値がメモリの中で何バイトを使うかを返す関数です（`std::mem::size_of_val`）。ふだんのアプリで Future の大きさを気にすることはあまりありませんが、「`.await` をまたぐと何が起きるか」を目で見るにはちょうどよい道具です。

よくあるエラーと読み方

エラー1: main に async を付けただけ（E0752）

`#[tokio::main]` を書き忘れて、`main` に `async` だけを付けた場合です。

```
async fn main() {
    println!("こんにちは");
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
error[E0752]: `main` function is not allowed to be `async`
 --> src\bin\e01_async_main.rs:1:1
  |
1 | async fn main() {
  | ^^^^^^^^^^^^^^^ `main` function is not allowed to be `async`

For more information about this error, try `rustc --explain E0752`.
error: could not compile `async_lab` (bin "e01_async_main") due to 1 previous error
```

- **何が起きたか:**「`main` 関数は `async` にできません」というエラーです。`async fn main` は Future を返しますが、`main` を呼ぶ OS は、その Future を poll する方法を知りません。
- **直し方:** `main` の上の行に `#[tokio::main]` を足します。これで、Tokio のランタイムが Future を動かしてくれる、ふつうの `main` に書き換わります。

エラー2: async ではない関数の中で .await（E0728）

```rust
use std::time::Duration;

// async が付いていない、ふつうの関数
fn load_note() -> String {
    tokio::time::sleep(Duration::from_millis(100)).await;
    String::from("ノート")
}

#[tokio::main]
async fn main() {
    println!("{}", load_note());
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
error[E0728]: `await` is only allowed inside `async` functions and blocks
 --> src\bin\e01_await_in_sync.rs:5:52
  |
4 | fn load_note() -> String {
  | ------------------------ this is not `async`
5 |     tokio::time::sleep(Duration::from_millis(100)).await;
  |                                                    ^^^^^ only allowed inside
`async` functions and blocks

For more information about this error, try `rustc --explain E0728`.
error: could not compile `async_lab` (bin "e01_await_in_sync") due to 1 previous error
```

- **何が起きたか:**「`await` は `async` の関数かブロックの中でしか使えません」というエラーです。`this is not async` の線が、`async` が付いていない関数の行を指しています。
- **直し方:** `fn load_note()` を `async fn load_note()` にします。すると `load_note()` は Future を返すようになるので、呼ぶ側も `load_note().await` に直します。

> **ヒント:** 関数の中で `.await` を使うと、その関数も `async` にする必要があり、その関数を呼ぶ側でも `.await` が必要になり…と、`async` は呼び出し元へ向かって広がっていきます。いちばん外側は `#[tokio::main]` の `main` で受け止めます。同期の関数から async の処理をどうしても呼びたいときの方法と、その落とし穴は UNIT 02 で扱います。

エラー3: .await の付け忘れ（E0308）

```
async fn count_notes() -> u32 {
    3
}

#[tokio::main]
async fn main() {
    // .await を付け忘れた
    let count: u32 = count_notes();
    println!("ノートは {count} 件");
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
error[E0308]: mismatched types
 --> src\bin\e01_forgot_await.rs:8:22
  |
8 |     let count: u32 = count_notes();
  |                ---   ^^^^^^^^^^^^^ expected `u32`, found future
  |                |
  |                expected due to this
  |
note: calling an async function returns a future
 --> src\bin\e01_forgot_await.rs:8:22
  |
8 |     let count: u32 = count_notes();
  |                      ^^^^^^^^^^^^^
help: consider `await`ing on the `Future`
  |
8 |     let count: u32 = count_notes().await;
  |                                   ++++++

For more information about this error, try `rustc --explain E0308`.
error: could not compile `async_lab` (bin "e01_forgot_await") due to 1 previous error
```

- **何が起きたか:** 「型が合いません。`u32` が来るはずなのに、future が来ました」というエラーです。`note:` の行が「async 関数を呼ぶと future が返ります」と、原因をそのまま教えてくれています。
- **直し方:** `help:` のとおり、`count_notes().await` にします。`++++++` の印は「ここに `.await` を足して」という意味です。

警告: 結果を使わない Future の .await 忘れ（unused implementer of Future）

いちばん危ないのは、エラーにならずに**ビルドが通ってしまう**場合です。

```
async fn save_note() {
    println!("保存しました");
}

#[tokio::main]
async fn main() {
    // .await を付け忘れた（結果も使っていない）
    save_note();
    println!("終わり");
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
warning: unused implementer of `Future` that must be used
 --> src\bin\e01_unused_future.rs:8:5
  |
8 |     save_note();
  |     ^^^^^^^^^^^
  |
  = note: futures do nothing unless you `.await` or poll them
  = note: `#[warn(unused_must_use)]` (part of `#[warn(unused)]`) on by default

warning: `async_lab` (bin "e01_unused_future") generated 1 warning
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.70s
     Running `target\debug\e01_unused_future.exe`
終わり
```

- **何が起きたか:** プログラムは動きましたが、表示は「終わり」だけで、「**保存しました」が出ていません。**`save_note()` は Future を作っただけで、誰も `.await` しなかったので、中身が1行も動かずに捨てられました。`note: futures do nothing unless you .await or poll them`（Future は `.await` か poll しない限り何もしません）が、そのことを教えています。
- **直し方:** `save_note().await;` にします。
- **見落とさない工夫:** 警告は、ほかの出力に紛れて見落としがちです。`cargo clippy -- -D warnings` を使うと、警告を**エラーとして扱い**、ビルドを失敗させてくれます。

```
cargo clippy --bin e01_unused_future -- -D warnings
```

```
    Checking async_lab v0.1.0 (...\async_lab)
error: unused implementer of `std::future::Future` that must be used
 --> src\bin\e01_unused_future.rs:8:5
  |
8 |     save_note();
  |     ^^^^^^^^^^^
  |
  = note: futures do nothing unless you `.await` or poll them
  = note: `-D unused-must-use` implied by `-D warnings`
  = help: to override `-D warnings` add `#[allow(unused_must_use)]`

error: could not compile `async_lab` (bin "e01_unused_future") due to 1 previous error
```

> **注意:**「保存したつもりで保存されていない」は、本番では気づくのがとても遅れる種類のバグです。RustNote の自動チェック（ `Rust/rustnote/ci.yml` ）でも、`cargo clippy --all-targets --locked -- -D warnings` を実行して、警告が1つでもあれば失敗にしています。

RustNote ではここで使う

RustNote の入口 `Rust/rustnote/src/main.rs` は、この単元で学んだ形そのものです（説明に関係する部分を抜き出しています）。

```rust
#[tokio::main]
async fn main() -> anyhow::Result<()> {
    // .env が無くてもエラーにしない（本番では環境変数を直接設定する）
    dotenvy::dotenv().ok();

    let config = Config::from_env().context("設定を読み込めませんでした")?;

    //  （アセットの読み込みは省略）

    let db = Db::open(&config)
        .await
        .with_context(|| format!("DB（{}）を開けませんでした", config.database_url))?;
    let applied = db::migrate(&db)
        .await
        .context("マイグレーションに失敗しました")?;

    //  （起動のログとルーターの組み立ては省略）

    topcoat::start(router)
        .await
        .context("サーバーを起動できませんでした（HOST / PORT を確認してください）")?;
    Ok(())
}
```

- `#[tokio::main]` の `async fn main` で、Tokio のランタイムの上でアプリ全体を動かしています。
- `Db::open(&config).await` は「DB を開き終わるまで待つ」、`db::migrate(&db).await` は「表の準備が終わるまで待つ」です。どちらも前の結果を次で使うので、**順番に** `.await` しています（例1-2の解説4）。
- `.await` の後に続く `.context(...)?` は、「待ち終わって `Result` が出てきたら、エラーなら説明を付けて返す」という意味です。`.await` が先、`?` が後、の順番で読みます。
- 最後の `topcoat::start(router).await` は、**サーバーが止まるまで**終わりません。サーバーは、ここで待ちながら、届いたリクエストを1つずつ別の仕事（タスク）として動かし続けます。そのしくみが、次の UNIT 02 のテーマです。

ノートを読む処理（`src/repository/notes.rs` の `find`）も、`.await` の連続です。

```
async fn find(&self, owner: UserId, id: NoteId) -> Result<Option<Note>, RepoError> {
    let conn = self.db.connect().await?;
    let sql = format!("SELECT {NOTE_COLUMNS} FROM notes WHERE id = ?1 AND user_id = ?
2");
    let mut rows = conn
        .query(&sql, libsql::params![id.get(), owner.get()])
        .await?;
    match rows.next().await? {
        Some(row) => Ok(Some(row_to_note(&row)?)),
        None => Ok(None),
    }
}
```

DB につなぐ・SQL を送る・1行目を受け取る、の3か所で `.await` しています。DB の返事を待っている間、同じサーバーの中では、ほかの人のリクエストが進んでいます。これが、RustNote が1つのプログラムで大勢の利用者をさばける理由です。

練習問題

問1: 次のプログラムを実行すると、何がどの順番で表示されるでしょうか。実行する前に予想してください（`src/bin/u01_q1.rs`）。

```
// 番号を受け取り、始めたことを表示して、10 倍した値を返す
async fn job(number: u32) -> u32 {
    println!("  仕事{number} を始めます");
    number * 10
}

#[tokio::main]
async fn main() {
    let first = job(1);
    let second = job(2);
    println!("札を2枚作りました");
    // 2枚目の札から先に .await する
    let b = second.await;
    let a = first.await;
    println!("結果: {a} {b}");
}
```

問2: 次のプログラムは、0.9 秒かかります。3つの問い合わせはお互いに関係がありません。`tokio::join!` を使って速くしてください。書き換えた後、全体で何秒になるかも予想してください（`src/bin/u01_q2.rs`）。

```rust
use std::time::{Duration, Instant};

// DB への問い合わせの代わり: 指定した時間だけ待ってから、名前を返す
async fn load(name: &str, millis: u64) -> String {
    tokio::time::sleep(Duration::from_millis(millis)).await;
    name.to_string()
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    let user = load("ユーザー", 200).await;
    let notes = load("ノート", 300).await;
    let tags = load("タグ", 400).await;
    println!(
        "{user}・{notes}・{tags} / {:.1} 秒",
        start.elapsed().as_secs_f64()
    );
}
```

```
ユーザー・ノート・タグ / 0.9 秒
```

問3: `tokio::try_join!` に渡した2つの Future が、**両方とも**失敗します。`load_user` は 0.3 秒後に、`count_notes` は 0.1 秒後に失敗します。表示される結果と時間を予想してください（`src/bin/u01_q3.rs`）。

```rust
use std::time::{Duration, Instant};

// 0.3 秒後に失敗する
async fn load_user() -> Result<String, String> {
    tokio::time::sleep(Duration::from_millis(300)).await;
    Err(String::from("ユーザーを読めません"))
}

// 0.1 秒後に失敗する
async fn count_notes() -> Result<u32, String> {
    tokio::time::sleep(Duration::from_millis(100)).await;
    Err(String::from("ノートを数えられません"))
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    let result = tokio::try_join!(load_user(), count_notes());
    println!("{result:?} / {:.1} 秒", start.elapsed().as_secs_f64());
}
```

解答と解説

問1の解答:

```
札を2枚作りました
  仕事2 を始めます
  仕事1 を始めます
結果: 10 20
```

- `job(1)` と `job(2)` を呼んだ時点では、どちらの中身も動いていません。だから「札を2枚作りました」が最初です。
- 中身が動くのは `.await` したときなので、先に `.await` した `second`（仕事2）が先に始まります。**呼んだ順番ではなく、`.await` した順番**で動く、というのがポイントです。
- `a` は `first`（仕事1）の結果の 10、`b` は `second` の結果の 20 です。

問2の解答:

```rust
use std::time::{Duration, Instant};

// DB への問い合わせの代わり: 指定した時間だけ待ってから、名前を返す
async fn load(name: &str, millis: u64) -> String {
    tokio::time::sleep(Duration::from_millis(millis)).await;
    name.to_string()
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    // 3つの待ち時間を重ねる。全体の時間は、いちばん長い 400 ミリ秒で決まる
    let (user, notes, tags) = tokio::join!(
        load("ユーザー", 200),
        load("ノート", 300),
        load("タグ", 400)
    );
    println!(
        "{user}・{notes}・{tags} / {:.1} 秒",
        start.elapsed().as_secs_f64()
    );
}
```

`src/bin/u01_q2_answer.rs` として保存して実行した結果です。

```
ユーザー・ノート・タグ / 0.4 秒
```

- 順番に待つと 0.2 + 0.3 + 0.4 = 0.9 秒、`join!` で重ねると、いちばん長い 0.4 秒です。

- `tokio::join!` の結果は、渡した順番どおりのタプル `(user, notes, tags)` で返ります。終わった順番（ユーザー → ノート → タグ）ではなく、**書いた順番**です。

問3の解答:

```
Err("ノートを数えられません") / 0.1 秒
```

- `try_join!` は、**最初に起きた**失敗を返します。0.1 秒の時点で `count_notes` が失敗したので、その `Err` がすぐに返りました。`load_user` は途中で捨てられ、0.3 秒の失敗は起きる前に終わっています。
- 書いた順番（`load_user` が先）ではなく、**時間の順番**で決まる点に注意しましょう。どちらのエラーが返るかは、それぞれの待ち時間しだいで変わります。

まとめ

- `async fn` を呼ぶと Future（仕事の札）が返るだけで、中身は `.await` したときに初めて動く。Future は poll されて `Pending` と `Ready` を返しながら進み、Waker で起こされる。
- お互いに関係のない待ちは `tokio::join!` / `tokio::try_join!` で重ねると、全体の時間が「足し算」から「いちばん長いもの」になる。これは並行で、CPU を2つ使う並列ではない。
- `.await` の付け忘れはエラーにならず警告だけのことがあるので、`cargo clippy -- -D warnings` で止める。

UNIT 02: Tokio のランタイム・spawn・JoinHandle

この単元のゴール

- Tokio の**ランタイム**が持つ係（仕事を割り振るスケジューラー・タイマー・I/O）と、`#[tokio::main]` が中でしていることを説明できる
- `tokio::spawn` で**タスク**を作り、`JoinHandle`（ジョインハンドル）で結果を受け取れる。タスクが何本ものスレッドで同時に動くことを、実行結果で確かめられる
- タスクの中で起きた panic と、`abort` による取り消しを、`JoinError` で見分けられる
- `tokio::spawn` に渡す仕事が `'static` と `Send` でなければならない理由を説明し、E0373 と「future cannot be sent between threads safely」のエラーを直せる
- ランタイムの中で `block_on` を呼んではいけない理由を説明できる

たとえ話でつかむ

UNIT 01 のラーメン屋は、店員が1人でした。今度は、料理人が何人もいる大きな厨房を考えます。

- 厨房全体が**ランタイム**です。料理人の1人1人が、**ワーカースレッド**（仕事を進めるスレッド）です。
- 注文の伝票を厨房の「注文レール」に掛けることが、`tokio::spawn` です。レールに掛かった伝票1枚1枚が**タスク**です。

- 伝票を掛けた人には、**引換券**（番号札）が渡されます。これが `JoinHandle` です。引換券を出せば、できあがった料理を受け取れます（`.await`）。
- 手の空いた料理人は、レールから次の伝票を取ります。自分の分が無くなったら、ほかの料理人の伝票も手伝います。麺をゆでる間（`Pending`）は、伝票を脇に置いて別の伝票を進めます。
- 店じまい（`main` の終わり）になると、レールに残った伝票は、途中でも全部捨てられます。

伝票は、どの料理人が手に取るか分かりません。途中で別の料理人に渡ることもあります。だから、伝票に添える物には2つの約束があります。

- **手渡ししてよい物（`Send`）であること。**第3巻 UNIT 10 で学んだ `Rc` のように「1人の手の中でしか使えない物」は添えられません。
- **店が開いている間ずっと使える物（`'static`）であること。**「隣の人から借りた物」を添えると、料理ができる前に貸し主が帰ってしまうかもしれません。借り物ではなく、自分の物（所有している値）を添えます。

> **用語:** タスク（task：ランタイムの上で動く、1つの独立した仕事。`tokio::spawn` で作る。中身は1つの Future）

> **用語:** ワーカースレッド（worker thread：ランタイムが用意した、タスクを poll して進めるスレッド。ふつうは CPU のコア数と同じ本数）

> **用語:** JoinHandle（ジョインハンドル： `tokio::spawn` が返す引換券。 `.await` すると、タスクの戻り値を `Result` に包んで受け取れる）

図で理解する

Tokio のランタイムの中は、次のようになっています。

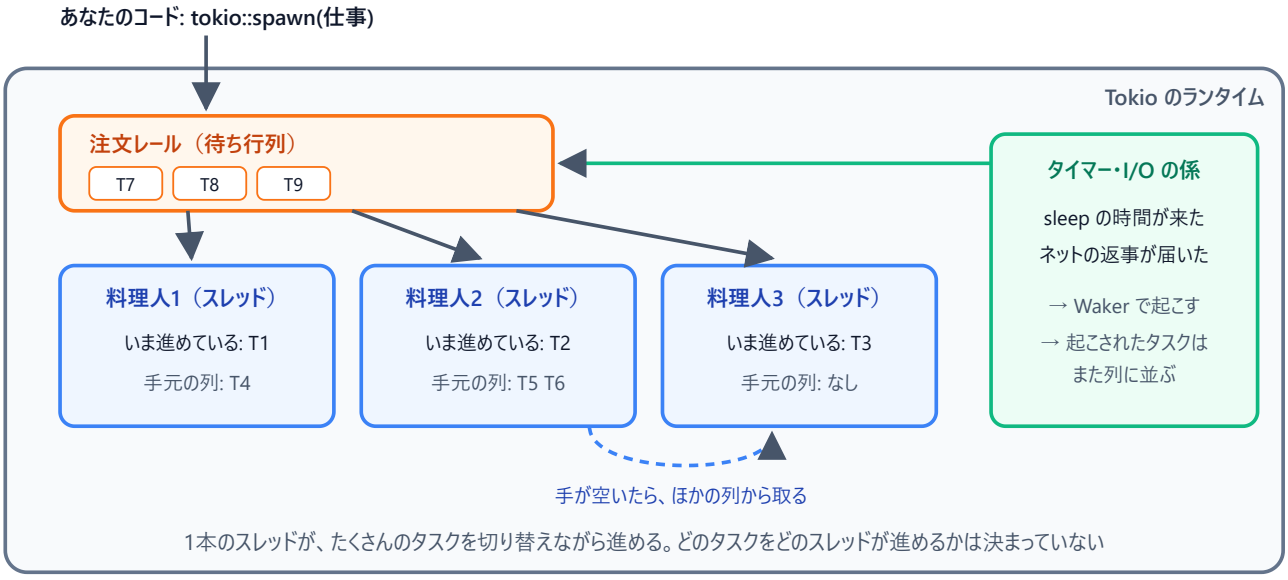


図5-6: Tokio のランタイムの中。spawn したタスクは待ち行列に並び、ワーカースレッドが取り出して poll する。待ちが終わったタスクは、タイマーや I/O の係に起こされて、また列に並ぶ

- **スケジューラー**（仕事の割り振り係）が、待ち行列に並んだタスクを、ワーカースレッドに配ります。手が空いたスレッドは、ほかのスレッドの列からもタスクを取ります。
- **タイマーと I/O の係**は、`sleep` の時間が来たときや、ネットワークの返事が届いたときに、待っていたタスクを Waker で起こします（UNIT 01 の図の④）。
- タスクは、`.await` で止まるたびに、**別のスレッドで続きが動くことがあります**。これが、タスクに `Send` が要る理由です。

次は、1つのタスクが作られてから、結果が届くまでの道すじです。

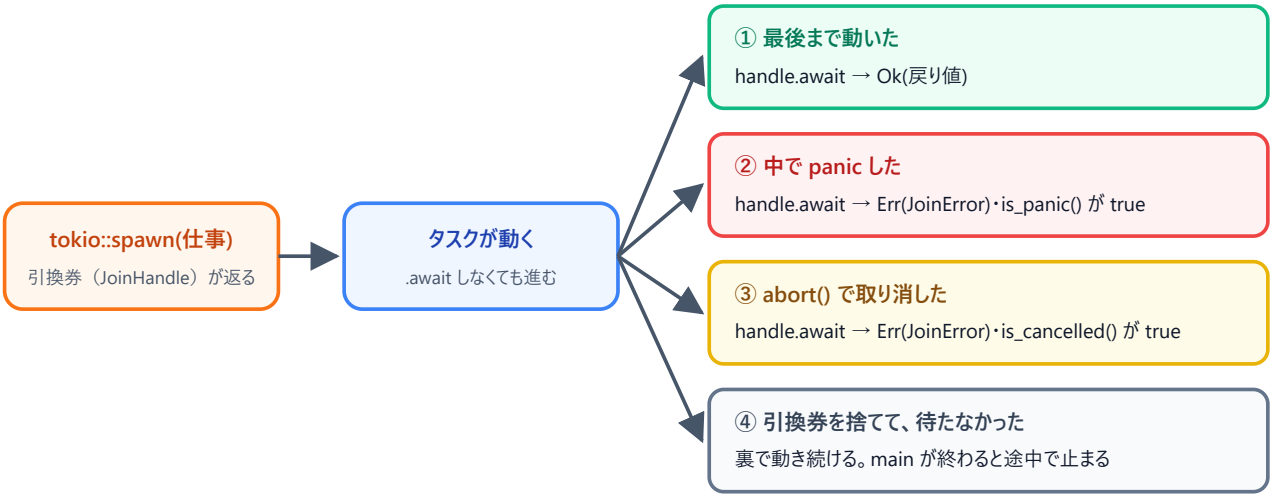


図5-7: タスクの一生。引換券を .await すると、最後まで動いたなら Ok、panic や取り消しなら JoinError が返る。引換券を捨てたタスクは裏で動き続けるが、main が終わると途中で止まる

UNIT 01 の Future と、この単元のタスクには、大きな違いが1つあります。

- **Future は、`.await` するまで動きません**（UNIT 01 の例1-1）。
- **タスクは、`tokio::spawn` した瞬間から動き始めます。**引換券を `.await` するのは「結果を受け取るため」で、「動かすため」ではありません。

第3巻 UNIT 10 で学んだ `std::thread::spawn`（OS のスレッド）と、この単元の `tokio::spawn`（Tokio のタスク）を比べると、次のようになります。

	`std::thread::spawn`（第3巻）	`tokio::spawn`（この単元）
作るもの	OS のスレッド	Tokio のタスク（中身は1つの Future）
1つあたりの重さ	重い。Rust の標準では、1本ごとに 2MB のスタックを用意する	軽い。Future の大きさ分（UNIT 01 の例1-5）だけ
待つとき	`thread::sleep` でスレッドごと眠る	`.await` で席を譲り、スレッドはほかのタスクを進める
結果の受け取り	`handle.join()`	`handle.await`
たくさん作ると	本数が増えるほど、メモリと切り替えの手間が重くなる	10万個でも動く（例2-5）
向いている仕事	長い計算・同期の処理	待ち時間の多い仕事（DB・ネットワーク）

まず動かす最小コード

例2-1: tokio::spawn でタスクを3つ作り、引換券で結果を受け取る

> **▼ このコードがやること（先に日本語で）**：0.3 秒・0.1 秒・0.2 秒待つタスクを3つ作ります。それぞれ、終わったときに「何秒の時点で、どのスレッドで終わったか」を表示します。その後、作った順番に引換券を `.await` して、戻り値を受け取ります。

```rust
use std::time::{Duration, Instant};

// 1つのタスクの仕事: 指定の時間だけ待ち、終わったら「どのスレッドで動いたか」を表示する
async fn work(number: u32, millis: u64, start: Instant) -> u32 {
    tokio::time::sleep(Duration::from_millis(millis)).await;
    let thread = std::thread::current();
    println!(
        "  タスク{number} が終わりました ({:.1} 秒 / {:?}) ",
        start.elapsed().as_secs_f64(),
        thread.id()
    );
    number * 100
}

#[tokio::main]
async fn main() {
    let start = Instant::now();
    // tokio::spawn でタスクを作ると、すぐに裏で動き始める。手元には JoinHandle（引換券）が残る
    let first = tokio::spawn(work(1, 300, start));
    let second = tokio::spawn(work(2, 100, start));
    let third = tokio::spawn(work(3, 200, start));
    println!("3つのタスクを作りました");

    // 引換券を .await すると、そのタスクの戻り値を受け取れる
    for handle in [first, second, third] {
        let value = handle.await.expect("タスクが panic しました");
        println!("受け取った値: {value}");
    }
    println!("合計 {:.1} 秒", start.elapsed().as_secs_f64());
}
```

`src/bin/u02_spawn.rs` として保存して実行します。

```
cargo run -q --bin u02_spawn
```

▼ 実行結果

```
3つのタスクを作りました
  タスク2 が終わりました (0.1 秒 / ThreadId(13))
  タスク3 が終わりました (0.2 秒 / ThreadId(13))
  タスク1 が終わりました (0.3 秒 / ThreadId(13))
受け取った値: 100
受け取った値: 200
受け取った値: 300
合計 0.3 秒
```

▼ コードを1つずつ分解して解説

解説1: spawn したタスクは、すぐに動き始める

- `tokio::spawn(work(1, 300, start))` は、`work(...)` が作った Future をタスクとして待ち行列に入れ、引換券（`JoinHandle<u32>`）をすぐに返します。`spawn` そのものは待ちません。
- 3つのタスクは、引換券を `.await` する前から動いています。「タスク2が終わりました」が 0.1 秒の時点で表示されているのは、そのためです。終わった順番は、待ち時間の短い順（2 → 3 → 1）です。
- 全体は、いちばん長い 0.3 秒で終わっています。UNIT 01 の `join!` と同じく、3つの待ちが重なりました。

解説2: 引換券の .await は、結果を受け取るため

- `for handle in [first, second, third]` で、作った順番に `.await` しています。タスク1（0.3 秒）を待っている間に、タスク2と3はとっくに終わっていますが、その結果はタスクの中に取っておかれます。
- だから「受け取った値」は、終わった順番ではなく、`.await` した順番（100 → 200 → 300）で表示されます。
- `handle.await` の結果は `Result<u32, JoinError>` です。タスクが panic していなければ `Ok`、panic していたら `Err` です。ここでは `expect` で中身を取り出しています（`Err` のときの例は例2-3）。

解説3: ThreadId は「どのスレッドで動いたか」の番号

- `std::thread::current().id()` は、今このコードを動かしているスレッドの番号です。
- この実行では、3つとも `ThreadId(13)` で終わりました。3つのタスクはほとんど「待っているだけ」なので、1本のスレッドで十分にさばけたのです。実行するたびに番号は変わり、別々のスレッドに分かれることもあります。

> **ヒント:** 同じ時刻に起こされたタスクが複数あると、それらは別々のスレッドで**本当に同時に**（並列に）動くことがあります。そのとき、どちらの `println!` が先に表示されるかは決まっていません。この巻の実行結果で、同じ待ち時間のタスクの表示の順番が教材と入れ替わるのは、このためです。

例2-2: #[tokio::main] を使わずに、ランタイムを自分で組み立てる

> **▼ このコードがやること（先に日本語で）：** `#[tokio::main]` を使わずに、ワーカースレッドが2本のランタイムを自分で作ります。その上で 0.1 秒待つタスクを4つ動かし、それぞれがどのスレッドで終わったかを表示します。最後に、4つの戻り値の合計を表示します。

```rust
use std::time::Duration;

// #[tokio::main] を使わずに、ランタイムを自分で組み立てる
fn main() -> std::io::Result<()> {
    let runtime = tokio::runtime::Builder::new_multi_thread()
        // ワーカースレッドを2本にする（既定は CPU のコア数）
        .worker_threads(2)
        // 時間（sleep・timeout）と I/O（ネットワーク）の機能を有効にする
        .enable_all()
        .build()?;

    // block_on: ここで渡した async ブロックが終わるまで、main のスレッドは待つ
    let total = runtime.block_on(async {
        let mut handles = Vec::new();
        for number in 1..=4 {
            handles.push(tokio::spawn(async move {
                tokio::time::sleep(Duration::from_millis(100)).await;
                let thread = std::thread::current();
                println!(
                    "タスク{number}: {} {:?}",
                    thread.name().unwrap_or("?"),
                    thread.id()
                );
                number * 10
            }));
        }
        let mut total = 0;
        for handle in handles {
            total += handle.await.unwrap();
        }
        total
    });
    println!("合計: {total}");
    Ok(())
}
```

`src/bin/u02_runtime.rs` として保存して実行します。

```
cargo run -q --bin u02_runtime
```

▼ 実行結果

```
タスク3: tokio-rt-worker ThreadId(3)
タスク2: tokio-rt-worker ThreadId(3)
タスク1: tokio-rt-worker ThreadId(3)
タスク4: tokio-rt-worker ThreadId(2)
合計: 100
```

▼ コードを1つずつ分解して解説

解説1: Builder で、ランタイムの形を決める

- `tokio::runtime::Builder::new_multi_thread()` は、「ワーカースレッドを何本も使うランタイム」を作る準備です。
- `.worker_threads(2)` で、ワーカースレッドの本数を2本にしています。実行結果に `ThreadId(3)` と `ThreadId(2)` の2種類しか出てこないのは、そのためです。指定しなければ、CPU のコア数と同じ本数になります。
- `.enable_all()` は、タイマー（`sleep` など）と I/O（ネットワーク）の係を有効にします。これを忘れると、`sleep` を使ったところで `A Tokio 1.x context was found, but timers are disabled.`（タイマーが有効になっていません）という panic になります。
- `.build()?` でランタイムができあがります。作るのに失敗することもあるので `Result` が返り、`?` で `main` のエラーとして返しています。

解説2: block_on は、同期の世界と async の世界の境目

- `runtime.block_on(Future)` は、「この Future が終わるまで、今のスレッドで待ち続ける」関数です。`main` はふつうの関数（同期の世界）なので、ここで async の世界に入ります。
- `block_on` が返した値（ここでは合計の `100`）は、ふつうの値として `main` で使えます。
- `#[tokio::main]` は、ちょうどこの形を自動で書いてくれる印です。`#[tokio::main] async fn main() { 中身 }` は、おおよそ次のように書き換えられます。

```
fn main() {
    tokio::runtime::Builder::new_multi_thread()
        .enable_all()
        .build()
        .unwrap()
        .block_on(async { /* 中身 */ })
}
```

解説3: スレッドの名前は tokio-rt-worker

- `thread.name()` は、スレッドの名前です。Tokio のワーカースレッドには `tokio-rt-worker` という名前が付いています。panic したときのメッセージにも、この名前が出てきます（例2-3）。
- 4つのタスクは、どれも 0.1 秒後に同時に起こされたので、表示の順番（3 → 2 → 1 → 4）は実行するたびに変わります。

> **ヒント:** `#[tokio::main(flavor = "current_thread")]` と書くと、ワーカースレッドを作らず、`main` のスレッド1本だけでタスクを動かすランタイムになります。テスト用の `#[tokio::test]` は、何も指定しなければこの1本の形です。並行には動きますが、並列にはなりません。

もう一歩: 実務に近い形

例2-3: タスクの中の panic と、取り消し（abort）を見分ける

> **▼ このコードがやること（先に日本語で）:** ①では、タスクの中で panic を起こします。②では、10 秒かかるタスクを作ってすぐに `abort`（取り消し）します。どちらも引換券を `.await` して、返ってきた `JoinError` が panic なのか取り消しなのかを調べます。最後に、`main` が最後まで動いたことを表示します。

```
use std::time::Duration;

#[tokio::main]
async fn main() {
    // ① タスクの中で panic する
    let panicking = tokio::spawn(async {
        let notes: Vec<&str> = Vec::new();
        // 空の Vec の 0 番目を読もうとして panic する
        notes[0].to_string()
    });
    match panicking.await {
        Ok(value) => println!("① 成功: {value}"),
        Err(error) => println!("① 失敗: {error} / is_panic = {}", error.is_panic()),
    }

    // ② 終わる前に abort（取り消し）する
    let slow = tokio::spawn(async {
        tokio::time::sleep(Duration::from_secs(10)).await;
        "10 秒の仕事が終わった"
    });
    slow.abort();
    match slow.await {
        Ok(value) => println!("② 成功: {value}"),
        Err(error) => println!("② 失敗: {error} / is_cancelled = {}",
error.is_cancelled()),
    }

    println!("main は最後まで動きました");
}
```

`src/bin/u02_join_error.rs` として保存して実行します。

```
cargo run -q --bin u02_join_error
```

▼ 実行結果

```
thread 'tokio-rt-worker' (18992) panicked at src\bin\u02_join_error.rs:9:14:
index out of bounds: the len is 0 but the index is 0
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
① 失敗: task 13 panicked with message "index out of bounds: the len is 0 but the index
is 0" / is_panic = true
② 失敗: task 14 was cancelled / is_cancelled = true
main は最後まで動きました
```

▼ コードを1つずつ分解して解説

解説1: タスクの panic は、そのタスクの中で止まる

- 最初の3行は、panic したときに Rust が表示するメッセージです。`thread 'tokio-rt-worker' (18992) panicked at ...:9:14:` は「ワーカースレッドの1本が、9行目14文字目で panic しました」という意味です。かっこの中の数字は OS のスレッドの番号で、実行するたびに変わります。
- 大事なのは、その後も `main` が動き続けていることです。Tokio は、タスクの中の panic を受け止めて、そのタスクだけを終わらせ、引換券に「panic しました」という `JoinError` を渡します。
- サーバーで言えば、1つのリクエストの処理で panic が起きても、サーバー全体は落ちずに、ほかの人のリクエストは処理され続けます。

解説2: JoinError の is_panic と is_cancelled

- `handle.await` が `Err(error)` を返したら、`error.is_panic()` で「panic したのか」、`error.is_cancelled()` で「取り消されたのか」を調べられます。
- `{error}` で表示すると、`task 13 panicked with message "..."` や `task 14 was cancelled` のように、タスクの番号と理由が出ます。この番号も、実行するたびに変わることがあります。

解説3: abort は、次の .await の区切りで止める

- `slow.abort()` は、タスクに「取り消し」を伝えます。タスクは、次に poll されるとき（`.await` の区切り）で止まり、中の Future は drop されます。10 秒待たずに、すぐ `is_cancelled = true` が返りました。
- 取り消しは、`.await` の区切りでしか起きません。`.await` の無い長い計算の途中では止まらないことに注意しましょう（UNIT 06 と UNIT 07 でくわしく扱います）。

> **注意:** panic が「サーバー全体は落ちない」からといって、panic に頼ってエラーを扱うのはやめましょう。panic したタスクで途中まで書き換えていたデータは、そのまま残ります。予想できる失敗（入力の間違い・ノートが無いなど）は、`Result` で返すのが基本です（第4巻 UNIT 05）。

例2-4: 引換券を捨てたタスクは、main が終わると途中で止まる

> **▼ このコードがやること（先に日本語で）**：0.1 秒ごとに5回表示する「裏のタスク」を作り、引換券は受け取らずに捨てます。`main` は 0.25 秒だけ待ってから終わります。裏のタスクが最後まで動くかどうかを確かめます。

```
use std::time::Duration;

#[tokio::main]
async fn main() {
    // JoinHandle を捨てて、結果を待たないタスク（切り離されたタスク）を作る
    tokio::spawn(async {
        for step in 1..=5 {
            println!("  裏のタスク: {step} 回目");
            tokio::time::sleep(Duration::from_millis(100)).await;
        }
        println!("  裏のタスク: 最後まで終わりました");
    });

    tokio::time::sleep(Duration::from_millis(250)).await;
    println!("main が終わります");
}
```

`src/bin/u02_detached.rs` として保存して実行します。

```
cargo run -q --bin u02_detached
```

▼ 実行結果

```
  裏のタスク: 1 回目
  裏のタスク: 2 回目
  裏のタスク: 3 回目
main が終わります
```

▼ コードを1つずつ分解して解説

解説1: 引換券を捨てても、タスクは止まらない

- `tokio::spawn(...)` の戻り値（引換券）を変数に入れずに捨てています。それでもタスクは、裏で動き続けます。引換券を捨てても、タスクは取り消されません（`abort` とは違います）。
- このような、誰も結果を待っていないタスクを「切り離されたタスク」（detached task）と呼びます。

解説2: main が終わると、ランタイムごと片付けられる

- 0.25 秒の時点で `main` が終わると、ランタイムが片付けられ、残っていたタスクは**途中でも全部捨てられます**。だから「4 回目」以降も、「最後まで終わりました」も表示されていません。
- 裏で「メールを送る」「ログを保存する」のような仕事をさせていたら、その仕事は黙って失われます。エラーも出ません。

> **注意:** 「最後までやり遂げてほしい仕事」を、引換券を捨てた `tokio::spawn` に任せてはいけません。終わるまで待つ方法は練習問題2で、サーバーを止めるときに処理中の仕事を待つ方法は UNIT 06 で、サーバーが落ちても仕事を失わない方法は UNIT 10 で学びます。

例2-5: タスクは軽い。10万個作っても約1秒

> **▼ このコードがやること（先に日本語で）:** 1 秒待つだけのタスクを10万個作り、作るのにかかった時間を表示します。その後、全部の引換券を `.await` して戻り値を足し合わせ、全体でかかった時間を表示します。

```
use std::time::{Duration, Instant};

#[tokio::main]
async fn main() {
    let start = Instant::now();
    let mut handles = Vec::new();
    // 1 秒待つだけのタスクを 10 万個作る
    for number in 0..100_000u64 {
        handles.push(tokio::spawn(async move {
            tokio::time::sleep(Duration::from_secs(1)).await;
            number
        }));
    }
    println!(
        "{} 個のタスクを作りました（{:.2} 秒）",
        handles.len(),
        start.elapsed().as_secs_f64()
    );

    // 全部のタスクの結果を受け取って、足し合わせる
    let mut total = 0;
    for handle in handles {
        total += handle.await.expect("タスクが panic しました");
    }
    println!(
        "全部終わりました: 合計 {total}（{:.2} 秒）",
        start.elapsed().as_secs_f64()
    );
}
```

`src/bin/u02_many_tasks.rs` として保存して実行します。

```
cargo run -q --bin u02_many_tasks
```

▼ 実行結果

```
100000 個のタスクを作りました（0.19 秒）
全部終わりました: 合計 4999950000（1.20 秒）
```

▼ コードを1つずつ分解して解説

解説1: 10万個の「待ち」が重なって、全体は約1秒

- 10万個のタスクを作るのに 0.19 秒、全部が終わるまでで 1.20 秒でした。1 秒の待ちが10万個重なったので、全体もほぼ 1 秒です。
- 同じことを OS のスレッド（`std::thread::spawn`）で10万本作ろうとすると、スタックの予約だけで 2MB × 10万 = 約200GB になります。OS のスレッドは、こんなに大量に作るための仕組みではありません。待ち時間の多い仕事を大量にさばくなら、タスクが向いています。
- 合計の 4999950000 は、0 から 99999 までを足した数です。全部のタスクの戻り値を、1つも取りこぼさずに受け取れています。

解説2: async move で、number をタスクへ移す

- `async move { ... }` の `move` は、「ブロックの中で使う外の変数を、借りるのではなく、タスクへ移す（所有権ごと渡す）」という意味です。`number` はループのたびに変わる変数なので、借りたままではタスクより先に消えてしまいます。
- `move` を付け忘れたときのエラーは、この後の「よくあるエラー1」で読みます。

> **注意:** 「タスクは軽い」とはいえ、外から来たリクエストの数だけ無制限にタスクを作ると、メモリや DB の接続が足りなくなります。同時に動かす数に上限を付ける方法は、UNIT 03 と UNIT 05 で学びます。

よくあるエラーと読み方

エラー1: move を付けずに外の変数を使う（E0373）

```
#[tokio::main]
async fn main() {
    let title = String::from("買い物メモ");
    // move を付けずに、外の変数 title を借りて使う
    let handle = tokio::spawn(async {
        println!("保存: {title}");
    });
    handle.await.unwrap();
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
error[E0373]: async block may outlive the current function, but it borrows `title`,
which is owned by the current function
 --> src\bin\e02_spawn_borrow.rs:5:31
  |
5 |     let handle = tokio::spawn(async {
  |                               ^^^^^ may outlive borrowed value `title`
6 |         println!("保存: {title}");
  |                          ----- `title` is borrowed here
  |
  = note: async blocks are not executed immediately and must either take a reference or
ownership of outside variables they use
help: to force the async block to take ownership of `title` (and any other referenced
variables), use the `move` keyword
  |
5 |     let handle = tokio::spawn(async move {
  |                                     ++++

For more information about this error, try `rustc --explain E0373`.
error: could not compile `async_lab` (bin "e02_spawn_borrow") due to 1 previous error
```

- **何が起きたか:**「async ブロックは、今の関数より長生きするかもしれないのに、今の関数が持っている `title` を借りています」というエラーです。タスクは `main` とは別に動くので、`main` の `title` が先に消えても動き続けるかもしれません。借り物を持ったまま貸し主より長生きするのは、第3巻で学んだ借用のルールに反します。
- `tokio::spawn` は、渡す仕事に `'static`（「借り物を持っていない」、つまり、いつまでも使える）を求めます。エラー2の出力にある `F: Future + Send + 'static` が、その条件です。
- **直し方:** `help:` のとおり `async move` にして、`title` の持ち主をタスクへ移します。

```rust
#[tokio::main]
async fn main() {
    let title = String::from("買い物メモ");
    // move を付けて、title の持ち主をタスクへ移す
    let handle = tokio::spawn(async move {
        println!("保存: {title}");
    });
    handle.await.unwrap();
}
```

```
保存: 買い物メモ
```

（`src/bin/u02_fix_move.rs` として実行した結果です。）

エラー2: Rc を持ったまま .await する（future cannot be sent between threads safely）

```rust
use std::rc::Rc;
use std::time::Duration;

#[tokio::main]
async fn main() {
    let handle = tokio::spawn(async {
        // Rc はスレッドをまたいで渡せない（Send ではない）型
        let title = Rc::new(String::from("買い物メモ"));
        // Rc を持ったまま .await する
        tokio::time::sleep(Duration::from_millis(10)).await;
        println!("保存: {title}");
    });
    handle.await.unwrap();
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
error: future cannot be sent between threads safely
   --> src\bin\e02_spawn_rc.rs:6:18
    |
  6 |       let handle = tokio::spawn(async {
    |  __________________^
  7 | |         // Rc はスレッドをまたいで渡せない（Send ではない）型
  8 | |         let title = Rc::new(String::from("買い物メモ"));
...   |
 11 | |         println!("保存: {title}");
 12 | |     });
    | |______^ future created by async block is not `Send`
    |
    = help: within `{async block@src\bin\e02_spawn_rc.rs:6:31: 6:36}`, the trait `Send`
is not implemented for `Rc<String>`
note: future is not `Send` as this value is used across an await
   --> src\bin\e02_spawn_rc.rs:10:55
    |
  8 |         let title = Rc::new(String::from("買い物メモ"));
    |             ----- has type `Rc<String>` which is not `Send`
  9 |         // Rc を持ったまま .await する
 10 |         tokio::time::sleep(Duration::from_millis(10)).await;
    |                                                       ^^^^^ await occurs here,
with `title` maybe used later
note: required by a bound in `tokio::spawn`
   --> ...\tokio-1.53.3\src\task\spawn.rs:176:21
    |
174 |     pub fn spawn<F>(future: F) -> JoinHandle<F::Output>
    |            ----- required by a bound in this function
175 |     where
176 |         F: Future + Send + 'static,
    |                     ^^^^ required by this bound in `spawn`

error: could not compile `async_lab` (bin "e02_spawn_rc") due to 1 previous error
```

エラーが長いので、上から順に区切って読みます。

- `future cannot be sent between threads safely` / `future created by async block is not Send`：「この Future は、スレッドの間で安全に受け渡せません。async ブロックが作った Future が `Send` ではありません」。
- `the trait Send is not implemented for Rc<String>`：原因の型は `Rc<String>` です。第3巻 UNIT 10 で学んだとおり、`Rc` は `Send` ではありません。
- `future is not Send as this value is used across an await`：「この値が `.await` をまたいで使われているので、Future が `Send` になりません」。`title` が8行目で作られ、10行目の `.await` の後（11行目）でも使われています。UNIT 01 の例1-5で見たとおり、`.await` をまたぐ変数は Future の中にしまわれるので、Future 全体が `Send` でなくなりました。

- `required by a bound in tokio::spawn` / `F: Future + Send + 'static`：`tokio::spawn` の定義で、渡す Future に `Send` が求められている、という出どころの説明です。タスクは `.await` のたびに別のスレッドで続きが動くかもしれないので、この条件があります。

直し方: スレッドをまたいで共有したいなら、`Rc` の代わりに `Arc` を使います（第3巻 UNIT 10）。

```
use std::sync::Arc;
use std::time::Duration;

#[tokio::main]
async fn main() {
    let handle = tokio::spawn(async {
        // Arc はスレッドをまたいで渡せる（Send な）型
        let title = Arc::new(String::from("買い物メモ"));
        tokio::time::sleep(Duration::from_millis(10)).await;
        println!("保存: {title}");
    });
    handle.await.unwrap();
}
```

```
保存: 買い物メモ
```

（`src/bin/u02_fix_rc.rs` として実行した結果です。）

> **ヒント**: そもそも共有する必要がないなら、ふつうの `String` で十分です。また、`Rc` を `.await` の**前**に使い終わって `{ }` の中で消しておけば、Future の中にしまわれないので、`Send` のエラーは出ません。「`.await` をまたいで持っている値は何か？」を確かめるのが、このエラーを読むコツです。UNIT 05 では、同じ形のエラーが `MutexGuard`（鍵）で起きます。

エラー3: ランタイムの中で block_on を呼ぶ（Cannot start a runtime from within a runtime）

コンパイルは通るのに、動かすと panic する例です。

```rust
// 同期の関数の中で、非同期の処理を「待ちきろう」とする
fn load_title() -> String {
    let runtime = tokio::runtime::Runtime::new().unwrap();
    runtime.block_on(async { String::from("買い物メモ") })
}

#[tokio::main]
async fn main() {
    // すでにランタイムの中（async fn main の中）から呼んでしまう
    let title = load_title();
    println!("{title}");
}
```

```
   Compiling async_lab v0.1.0 (...\async_lab)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.72s
     Running `target\debug\e02_block_on.exe`

thread 'main' (22388) panicked at ...\tokio-
1.53.3\src\runtime\scheduler\multi_thread\mod.rs:91:9:
Cannot start a runtime from within a runtime. This happens because a function (like
`block_on`) attempted to block the current thread while the thread is being used to
drive asynchronous tasks.
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
error: process didn't exit successfully: `target\debug\e02_block_on.exe` (exit code:
101)
```

- **何が起きたか:**「ランタイムの中で、ランタイムを始めることはできません。`block_on` のような関数が、async のタスクを動かしているスレッドを止めて待とうとしました」という panic です。
- `block_on` は「終わるまでこのスレッドで待ち続ける」関数でした（例2-2）。ワーカースレッドがそこで待ち続けると、そのスレッドで動くはずのほかのタスクが全部止まってしまいます。Tokio はそれを防ぐために、わざと panic して知らせています。
- 今動いているランタイムを借りる `tokio::runtime::Handle::current().block_on(...)` に書き換えても、同じ panic になります（`e02_block_on_handle.rs` で確かめた結果も、同じメッセージでした）。
- **直し方:** 待つ処理を含む関数は `async fn` にして、呼ぶ側で `.await` します。async の世界の中では、`block_on` を使いません。

```rust
// 待つ処理を含む関数は、async fn にして .await で待つ
async fn load_title() -> String {
    String::from("買い物メモ")
}

#[tokio::main]
async fn main() {
    let title = load_title().await;
    println!("{title}");
}
```

```
買い物メモ
```

（`src/bin/u02_fix_block_on.rs` として実行した結果です。）

> **ヒント:** `block_on` を使ってよいのは、`main` の入口（例2-2）のように「まだ async の世界の外にいる」場所だけです。逆に、async の世界から、時間のかかる同期の処理を呼びたいときは、`tokio::task::spawn_blocking` を使います（UNIT 07）。

RustNote ではここで使う

RustNote が使う Web フレームワーク Topcoat は、`topcoat::start(router).await` の中で、ブラウザからの**接続を1つ受け付けるたびに、タスクを1つ作って**います。Topcoat 0.10.0 のソース（`topcoat-router-0.10.0/src/service.rs` の `internal_serve`）から、その部分を抜き出すと次のとおりです（コメントは日本語に置き換え、関係の無い部分は省略しています）。

```rust
loop {
    // 新しい接続を待つ。止める合図（Ctrl+C）が来たら、ループを抜ける
    let accepted = tokio::select! {
        accepted = listener.accept() => accepted,
        () = &mut shutdown => break,
    };
    let (stream, remote_addr) = accepted?;
    // （省略）

    // 1つの接続を、1つのタスクで処理する
    tokio::spawn(async move {
        // （この接続のリクエストを読み、ルーターに渡して、答えを書き戻す。省略）
    });
}
```

- 接続ごとに `tokio::spawn` するので、Aさんのリクエストが DB の返事を待っている間も、Bさん・Cさんのリクエストは別のタスクとして進みます。どれかのタスクで panic が起きても、ほかの接続は影響を受けません（例2-3）。
- `tokio::select!` は「どちらか先に起きたほうを選ぶ」書き方で、UNIT 03 で学びます。止める合図を受けたときに、処理中の接続を待ってから終わる部分は、UNIT 06 で読みます。

タスクは、どのワーカースレッドで動くか分かりません。そのため、リクエストを処理するコードが使う共有の部品は、スレッドをまたいで使えなければなりません。RustNote の `AppState`（`src/state.rs`）が、保存先を次のように持っているのはそのためです。

```
#[derive(Clone)]
pub struct AppState {
    /// DB そのもの（GET /health の確認に使う）
    pub db: Db,
    pub users: Arc<dyn UserRepository + Send + Sync>,
    pub notes: Arc<dyn NoteRepository + Send + Sync>,
    pub sessions: Arc<dyn SessionRepository + Send + Sync>,
}
```

- `Arc` は、たくさんのタスクで同じものを共有するための入れ物です（第3巻 UNIT 10）。`Rc` では、エラー2と同じエラーになります。
- `dyn NoteRepository + Send + Sync` は、「中身は `NoteRepository` を実装した何かで、別のスレッドへ渡してよく（`Send`）、何本ものスレッドから同時に使ってよい（`Sync`）もの」という意味です（第4巻 UNIT 04 の `dyn`）。

もう1つ、パスワードを確かめる `src/auth.rs` の `verify_password` も、引換券の結果を受け取る形をしています。

```
    let joined = tokio::task::spawn_blocking(move || {
        // （パスワードとハッシュを照らし合わせる。省略）
    })
    .await;
    match joined {
        Ok(Ok(matches)) => Ok(matches),
        Ok(Err(error)) => Err(AppError::internal(error)),
        Err(error) => Err(AppError::internal(error)),
    }
```

`spawn_blocking` は UNIT 07 で学ぶ「重い計算用の spawn」です。引換券を `.await` した結果が `Result` で、その中身もまた `Result` なので、`Ok(Ok(..))`・`Ok(Err(..))`・`Err(..)` の3つに分けています。この形は、練習問題3で自分でも書きます。

練習問題

問1: 次のプログラムは、コンパイルエラー（E0373）になります。「hanako さんのノート1」〜「3」を表示し、最後に「持ち主: hanako」と表示するように直してください（`src/bin/u02_q1.rs`）。

```
#[tokio::main]
async fn main() {
    let owner = String::from("hanako");
    let mut handles = Vec::new();
    for number in 1..=3 {
        handles.push(tokio::spawn(async {
            format!("{owner} さんのノート{number}")
        }));
    }
    for handle in handles {
        println!("{}", handle.await.unwrap());
    }
}
```

> **ヒント:** `async move` にするだけでは、別のエラー（E0382）になります。ループの1回目で `owner` がタスクへ移ってしまい、2回目に使えなくなるからです。

問2: 例2-4の `u02_detached.rs` を書き換えて、裏のタスクが「最後まで終わりました」と表示してから、`main` が終わるようにしてください（`src/bin/u02_q2.rs`）。

問3: 次の `load_user` を、ユーザー 1・2・3 のそれぞれについて `tokio::spawn` で動かします。引換券を `.await` した結果を、「成功」「アプリのエラー」「タスクが失敗（panic）」の3つに分けて表示してください。`handle.await` の型が何になるかも考えましょう（`src/bin/u02_q3.rs`）。

```
use std::time::Duration;

// 1 なら成功、2 ならアプリのエラー、それ以外は panic する
async fn load_user(user_id: i64) -> Result<String, String> {
    tokio::time::sleep(Duration::from_millis(100)).await;
    match user_id {
        1 => Ok(String::from("hanako")),
        2 => Err(format!("ユーザー {user_id} はいません")),
        _ => panic!("ユーザー {user_id} で想定外のことが起きました"),
    }
}
```

解答と解説

問1の解答:

```rust
#[tokio::main]
async fn main() {
    let owner = String::from("hanako");
    let mut handles = Vec::new();
    for number in 1..=3 {
        // タスクごとに自分用の写しを作り、それを move で持っていく
        let owner = owner.clone();
        handles.push(tokio::spawn(async move {
            format!("{owner} さんのノート{number}")
        }));
    }
    for handle in handles {
        println!("{}", handle.await.unwrap());
    }
    // 元の owner は、まだ main で使える
    println!("持ち主: {owner}");
}
```

```
hanako さんのノート1
hanako さんのノート2
hanako さんのノート3
持ち主: hanako
```

- 元のコードのエラーは、`owner` と `number` の2つについて E0373 が出ます（どちらも借りているため）。
- ヒントのとおり、`async move` にするだけだと、次のエラーになります。

```
   Compiling async_lab v0.1.0 (...\async_lab)
error[E0382]: use of moved value: `owner`
 --> src\bin\e02_q1_move_only.rs:6:35
  |
3 |     let owner = String::from("hanako");
  |         ----- move occurs because `owner` has type `String`, which does not
implement the `Copy` trait
4 |     let mut handles = Vec::new();
5 |     for number in 1..=3 {
  |     ------------------ inside of this loop
6 |         handles.push(tokio::spawn(async move {
  |                                   ^^^^^^^^^^ value moved here, in previous
iteration of loop
7 |             format!("{owner} さんのノート{number}")
  |                      ----- use occurs due to use in coroutine
  |
help: consider cloning the value before moving it into the closure
  |
6 ~         let value = owner.clone();
7 ~         handles.push(tokio::spawn(async move {
8 ~             format!("{value} さんのノート{number}")
  |

For more information about this error, try `rustc --explain E0382`.
error: could not compile `async_lab` (bin "e02_q1_move_only") due to 1 previous error
```

- `value moved here, in previous iteration of loop` は「ループの前の回で、もう移してしまいました」という意味です。`coroutine`（コルーチン）は、async ブロックのコンパイラの中での呼び名です。
- `help:` のとおり、`move` の前に `clone()` で写しを作り、写しのほうをタスクへ移します。解答では、写しにも同じ `owner` という名前を付けて（シャドーイング：第2巻 UNIT 01）、`format!` の中を書き換えずに済ませています。`number` は `Copy` な整数なので、`move` だけで写されます。
- 長い文字列や大きなデータを何十個ものタスクで共有するなら、`clone` で毎回写す代わりに、`Arc<String>` にして `Arc::clone` で「持ち手」だけを増やす方法もあります（第3巻 UNIT 10）。

問2の解答:

```rust
use std::time::Duration;

#[tokio::main]
async fn main() {
    // JoinHandle を変数に取っておく
    let background = tokio::spawn(async {
        for step in 1..=5 {
            println!("  裏のタスク: {step} 回目");
            tokio::time::sleep(Duration::from_millis(100)).await;
        }
        println!("  裏のタスク: 最後まで終わりました");
    });

    tokio::time::sleep(Duration::from_millis(250)).await;
    // main を終える前に、裏のタスクが終わるのを待つ
    background.await.expect("裏のタスクが panic しました");
    println!("main が終わります");
}
```

```
  裏のタスク: 1 回目
  裏のタスク: 2 回目
  裏のタスク: 3 回目
  裏のタスク: 4 回目
  裏のタスク: 5 回目
  裏のタスク: 最後まで終わりました
main が終わります
```

- 引換券を捨てずに `background` に取っておき、`main` の最後で `.await` しました。`main` は、裏のタスクが終わるまでそこで待つので、5回目まで表示されます。
- `.await` の位置が 0.25 秒の `sleep` の後でも、裏のタスクはその間ずっと動いています。引換券の `.await` は「結果を受け取るため」（解説2）だからです。

問3の解答:

```rust
use std::time::Duration;

// 1 なら成功、2 ならアプリのエラー、それ以外は panic する
async fn load_user(user_id: i64) -> Result<String, String> {
    tokio::time::sleep(Duration::from_millis(100)).await;
    match user_id {
        1 => Ok(String::from("hanako")),
        2 => Err(format!("ユーザー {user_id} はいません")),
        _ => panic!("ユーザー {user_id} で想定外のことが起きました"),
    }
}

#[tokio::main]
async fn main() {
    for user_id in 1..=3 {
        let handle = tokio::spawn(load_user(user_id));
        // handle.await の型は Result<Result<String, String>, JoinError>
        match handle.await {
            Ok(Ok(name)) => println!("{user_id}: 成功 {name}"),
            Ok(Err(message)) => println!("{user_id}: アプリのエラー {message}"),
            Err(join_error) => println!("{user_id}: タスクが失敗 {join_error}"),
        }
    }
}
```

```
1: 成功 hanako
2: アプリのエラー ユーザー 2 はいません

thread 'tokio-rt-worker' (29704) panicked at src\bin\u02_q3.rs:9:14:
ユーザー 3 で想定外のことが起きました
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
3: タスクが失敗 task 15 panicked with message "ユーザー 3 で想定外のことが起きました"
```

- `handle.await` の型は `Result<Result<String, String>, JoinError>` です。外側の `Result` は「タスクが最後まで動いたか」、内側の `Result` は「`load_user` の処理が成功したか」を表します。
- `match` で `Ok(Ok(name))`・`Ok(Err(message))`・`Err(join_error)` の3つに分けると、「アプリのエラー」と「タスクそのものの失敗」を区別できます。RustNote の `verify_password` と同じ形です。
- `load_user(user_id)` はそれ自体が Future なので、`async move { ... }` で包まずに、そのまま `tokio::spawn` に渡せます。`user_id` は関数の引数として Future の中に移されています。

まとめ

- Tokio のランタイムは、ワーカースレッド・スケジューラー・タイマーと I/O の係でできている。`#[tokio::main]` は、ランタイムを作って `block_on` で `main` の中身を動かす形の書き換え。

- `tokio::spawn` したタスクは、`.await` しなくてもすぐ動き、引換券（`JoinHandle`）の `.await` で `Result<戻り値, JoinError>` を受け取る。引換券を捨てたタスクは、`main` が終わると途中で捨てられる。
- タスクに渡す仕事は `Send + 'static` でなければならない。外の変数は `move` で移し、共有は `Arc`、`.await` をまたいで `Rc` を持たない。ランタイムの中で `block_on` は使わない。

この巻のまとめ

この巻では、たくさんの仕事を同時にさばくサーバーの土台として、async の仕組みと Tokio のタスクから学び始めました。次のチェックリストで、できるようになったことを確かめてください。自信がない項目は、括弧の中の単元に戻って、コードを動かし直しましょう。表示の順番を予想してから動かすのを忘れずに。

async の土台

- ☐ `async fn` を呼ぶと Future（仕事の札）が返るだけで、中身は `.await` したときに初めて動くことを、実行結果で説明できる（UNIT 01）
- ☐ `tokio::time::sleep(...).await` と `std::thread::sleep` の違いを説明し、async の中では前者を使える（UNIT 01）
- ☐ お互いに関係のない待ちを `tokio::join!` で重ね、全体の時間が「足し算」から「いちばん長いもの」になることを示せる。並行と並列の違いを説明できる（UNIT 01）
- ☐ `tokio::try_join!` が最初に起きた失敗をすぐに返し、残りの Future を途中で捨てる（キャンセルする）ことを説明できる（UNIT 01）
- ☐ Future が poll されて `Pending` / `Ready` を返し、Waker で起こされて進む流れを図で説明できる。`.await` をまたぐ変数が Future の中にしまわれることを知っている（UNIT 01）
- ☐ E0752（`async fn main`）・E0728（`async` でない関数の `.await`）・E0308（`.await` の付け忘れ）と、`unused implementer of Future` の警告を読んで直せる。警告を `cargo clippy -- -D warnings` で止められる（UNIT 01）
- ☐ ランタイムの中身（ワーカースレッド・スケジューラー・タイマーと I/O の係）と、`#[tokio::main]` が `Builder` と `block_on` の形に書き換わることを説明できる（UNIT 02）
- ☐ `tokio::spawn` でタスクを作り、引換券（`JoinHandle`）の `.await` で `Result<戻り値, JoinError>` を受け取れる。タスクは spawn した瞬間から動くことを知っている（UNIT 02）
- ☐ `JoinError` の `is_panic()` と `is_cancelled()` で、panic と `abort` を見分けられる。`Result<Result<T, E>, JoinError>` を3つに分けて扱える（UNIT 02）
- ☐ 引換券を捨てたタスクが `main` の終わりで途中で捨てられることを知っていて、最後までやり遂げてほしい仕事は待つようにできる（UNIT 02）
- ☐ `tokio::spawn` に `Send + 'static` が要る理由を説明し、E0373・E0382（ループの中の `move`）・「future cannot be sent between threads safely」を、`move`・`clone`・`Arc` で直せる（UNIT 02）
- ☐ ランタイムの中で `block_on` を呼ぶと `Cannot start a runtime from within a runtime` で panic する理由を説明できる（UNIT 02）

> **お知らせ**: この版には UNIT 01・02 を収録しています。UNIT 03（JoinSet・select!）・UNIT 04（channel・backpressure）・UNIT 05（Mutex・RwLock・Semaphore）・UNIT 06（timeout・キャンセル・graceful shutdown）・UNIT 07（spawn_blocking）・UNIT 08（HTTP クライアント・再試行）・UNIT 09（tracing）・UNIT 10（バックグラウンドジョブ）は準備中で、次の更新から順に追加します。それまでの間、重い計算を逃がす `spawn_blocking` が RustNote でどう使われているかは、`Rust/rustnote/src/auth.rs` で確かめられます。

次の第6巻では、この巻で学んだタスクの上で動く Web フレームワーク Topcoat を使って、HTTP のリクエストと答えのしくみを学びます。

用語ミニ辞典

この巻で出てきた主な用語です。

用語	読み・意味	出てくる単元
async（非同期処理）	エイシンク。待ち時間の間に、ほかの仕事を進められるように書いた処理	巻頭・UNIT 01
Tokio	トーキョー。Future を動かすランタイムを提供するクレート	巻頭・UNIT 02
Future	フューチャー。「まだ終わっていない仕事」を表す値。`.await` か poll されるまで動かない	UNIT 01
`.await`	Future が終わるまで待つ印。待つ間は、ほかの仕事に席を譲る	UNIT 01
ランタイム（executor）	Future を poll して動かす係。実行係	UNIT 01・02
poll	ポール。Future に「進められるところまで進めて」と頼むこと	UNIT 01
Pending / Ready	poll の答え。「まだです」と「できました（値つき）」	UNIT 01
Waker	ウェイカー。準備ができた Future を、もう一度 poll するよう知らせる起こし係	UNIT 01
状態機械	「今どこまで進んだか」を状態として持ち、少しずつ進むしくみ。async fn はこれに作り変えられる	UNIT 01
pin	ピン。Future をメモリの中で動かないように留めること	UNIT 01
並行（concurrency）	コンカレンシー。複数の仕事を切り替えながら少しずつ進めること	UNIT 01
並列（parallelism）	パラレリズム。複数の CPU で、本当に同時に動かすこと	UNIT 01・02
`tokio::join!`	複数の Future を同時に進め、全部の結果をタプルで返す	UNIT 01
`tokio::try_join!`	`Result` を返す Future 用の `join!`。最初の失敗をすぐに返す	UNIT 01
キャンセル	Future が途中で drop され、二度と poll されなくなること	UNIT 01・02
タスク	ランタイムの上で動く、1つの独立した仕事。`tokio::spawn` で作る	UNIT 02
ワーカースレッド	ランタイムがタスクを進めるために用意したスレッド	UNIT 02
スケジューラー	待ち行列のタスクを、ワーカースレッドに割り振る係	UNIT 02
`tokio::spawn`	Future をタスクとして動かし始め、引換券を返す関数	UNIT 02
JoinHandle	ジョインハンドル。タスクの引換券。`.await` で結果を受け取る	UNIT 02
JoinError	タスクが panic したか、取り消されたことを表すエラー	UNIT 02
abort	アボート。タスクを取り消すこと。次の `.await` の区切りで止まる	UNIT 02

用語	読み・意味	出てくる単元
切り離されたタスク	引換券を捨てて、誰も結果を待っていないタスク（detached task）	UNIT 02
`block_on`	Future が終わるまで、今のスレッドで待ち続ける関数。async の世界の入口で使う	UNIT 02
`Send`	別のスレッドへ渡してよい、という型の印（第3巻 UNIT 10）	UNIT 02
`'static`	借り物を持たず、いつまでも使える、という印。`tokio::spawn` に渡す仕事に求められる	UNIT 02
`async move`	外の変数を借りずに、async ブロックの中へ移す書き方	UNIT 02
coroutine	コルーチン。コンパイラの中での、async ブロックの呼び名。エラーメッセージに出てくる	UNIT 02